Taller 3: Árboles de búsqueda

Saul Leonardo Cruz Romero[1], Isabella Hermosa Losada[1], Alejandro Macias Barrios[1]

[1] Departamento de Ingeniería de Sistemas, Pontificia Universidad Javeriana, Bogotá, Colombia .

Contributing authors: sa.cruz@javeriana.edu.co; ishermosa@javeriana.edu.co; amaciasb@javeriana.edu.co;

Resumen

En el Taller 3 de la clase de Estructuras de Datos se comparan tres estructuras de datos para almacenar y organizar placas de automóviles: un árbol AVL de implementación propia, un árbol rojinegro mediante la estructura `std::set` de la STL y un montículo utilizando las herramientas disponibles en C++.
Además de implementar las operaciones necesarias, se verifica que las tres estructuras conserven el mismo conjunto de elementos después de cada prueba. También se realiza el desarrollo gráfico paso a paso de las inserciones y eliminaciones solicitadas y se miden los tiempos acumulados de ejecución para comparar el comportamiento de cada estructura.

Keywords: árbol AVL, árbol rojinegro, montículo, estructuras de datos, C++

1. Introducción

Los árboles de búsqueda permiten almacenar información manteniendo una organización que facilita operaciones como la inserción, búsqueda y eliminación de elementos. Sin embargo, distintas estructuras pueden realizar estas operaciones de formas diferentes y presentar comportamientos distintos a medida que aumenta la cantidad de datos.

En este taller se trabaja con tres estructuras: un árbol AVL desarrollado a partir de una implementación propia realizada previamente a partir de las explicaciones del profesor, un árbol rojinegro representado mediante `std::set` y un montículo implementado utilizando mecanismos de la STL. Las tres estructuras procesan las mismas

operaciones para poder comparar sus resultados y verificar que mantengan el mismo conjunto de elementos.

El programa principal lee archivos de prueba que contienen operaciones `add` y `del`. Cada operación es aplicada sobre las tres estructuras y posteriormente se comparan las salidas obtenidas. Después de comprobar el funcionamiento de las implementaciones, se realizan pruebas de tiempo y se analizan las diferencias observadas entre ellas.

2. Desarrollo

2.1. Árbol AVL

Para el árbol AVL se utilizó la implementación propia desarrollada previamente. Esta implementación tuvo que adaptarse para que pudiera trabajar con la función genérica `LeerArbol` utilizada por el programa principal.

Los TODO #01, #02 y #03 corresponden a incluir la implementación del AVL, definir el tipo de árbol utilizado y crear la estructura dentro del `main`. Para las pruebas se utilizó `std::string` como tipo de dato, ya que las claves de los archivos corresponden a placas.

2.1.1. Adaptación de las operaciones de inserción y eliminación

La función `LeerArbol` utiliza los métodos `insert` y `erase` para procesar las operaciones `add` y `del`. Nuestra implementación del AVL ya contaba con los métodos `insertar` y `eliminar`, por lo que se agregaron métodos compatibles que delegan las operaciones sobre las funciones existentes.

```cpp
// el profe usa insert pero nosotros ya teniamos insertar
// entonces este solo conecta un nombre con el otro
template< class T >
bool ArbolAVL<T>::insert(T val) {
    return this->insertar(val);
}


// lo mismo pasa con erase y eliminar
template< class T >
bool ArbolAVL<T>::erase(T val) {
    return this->eliminar(val);
}
```

Figura 1 Adaptación de los métodos `insert` y `erase` del árbol AVL.

Con esta adaptación, el mismo mecanismo de lectura puede ejecutar inserciones y eliminaciones directamente sobre el AVL. Posteriormente, el árbol se pasa a `LeerArbol`, completando el TODO #04.

2.1.2. Recorrido inorden almacenado en una lista

El TODO #05 requiere guardar el recorrido inorden del AVL en una lista para poder compararlo posteriormente con las otras estructuras.

La implementación original ya permitía recorrer el árbol en inorden. Para el taller se agregó `inordenEnLista`, manteniendo el mismo orden de visita: primero el subárbol izquierdo, después el dato almacenado en el nodo y finalmente el subárbol derecho.

```
// nuevo para el taller
// es exactamente el mismo recorrido inorden
// solo que en vez de mostrar el dato lo guardamos en la lista
template< class T >
void NodoAVL<T>::inordenEnLista(std::list<T>& lista) {

    if (this->hijoIzq != NULL)
        this->hijoIzq->inordenEnLista(lista);

    lista.push_back(this->dato);

    if (this->hijoDer != NULL)
        this->hijoDer->inordenEnLista(lista);
}
```

Figura 2 Implementación del recorrido inorden almacenado en una lista.

De esta manera se obtiene una lista ordenada con los elementos almacenados en el AVL, que posteriormente se utiliza para verificar los resultados frente al árbol rojinegro y el montículo.

2.2. Pruebas iniciales

Antes de realizar las comparaciones de rendimiento se comprobó que el programa compilara correctamente y que las estructuras procesaran los archivos de prueba esperados.

2.2.1. Compilación

El programa se compiló utilizando el estándar C++11. La compilación final se realizó sobre el archivo fuente del taller y terminó correctamente.

```
isa21087@isa21087-Nitro-ANV15-51:~/Universidad/Estructura de Datos/Talleres Estr
ucturas/talleres2corte/taller_3_arboles_busqueda_archivos$ g++ -std=c++11 -o tal
ler_3_ordenamiento_busqueda taller_3_ordenamiento_busqueda.cxx
isa21087@isa21087-Nitro-ANV15-51:~/Universidad/Estructura de Datos/Talleres Estr
ucturas/talleres2corte/taller_3_arboles_busqueda_archivos$
```

Figura 3 Compilación final del programa utilizando C++11.

2.2.2. Prueba con diez inserciones

Se utilizó el archivo `01_solo_agregar_10.txt`, el cual contiene diez operaciones de inserción. Al finalizar la ejecución, las tres estructuras contenían los diez elementos y la comparación de sus salidas produjo coincidencia.

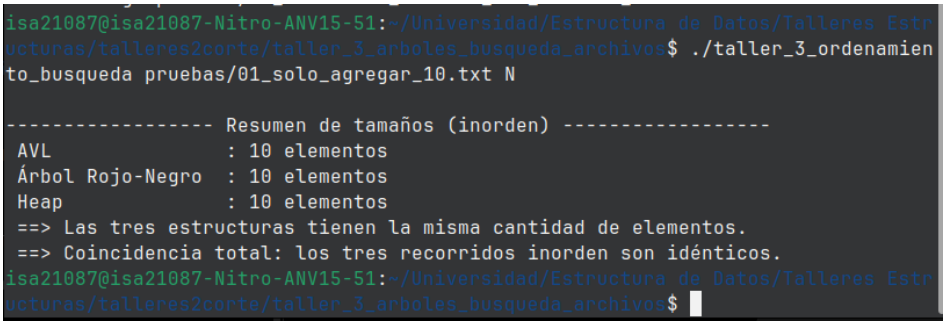


Figura 4 Resultado de la ejecución con el archivo de diez inserciones.

La secuencia ordenada obtenida fue:

`BBB060, DDD125, FFF190, GGG250, HHH310, JJJ375, MMM500, PPP625, TTT750, WWW875`

2.2.3. Prueba con inserciones y eliminaciones

También se ejecutó el archivo `07_agregar_10_quitar_5.txt`. En este caso se realizan primero diez inserciones y posteriormente cinco eliminaciones.

Al finalizar, cada estructura conserva cinco elementos y las salidas ordenadas coinciden.

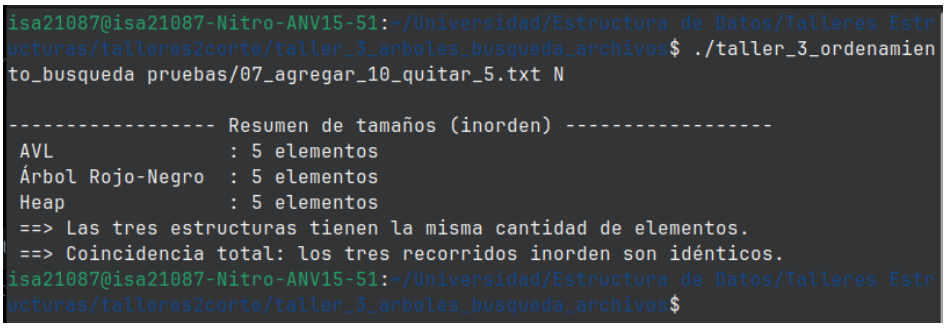


Figura 5 Resultado de la ejecución después de diez inserciones y cinco eliminaciones.

Los elementos restantes son:

`FFF190, HHH310, JJJ375, PPP625, WWW875`

Estas pruebas permiten comprobar que las operaciones de inserción y eliminación producen resultados consistentes antes de continuar con el análisis gráfico y las mediciones de tiempo.

2.3. Árbol rojinegro

Para representar el árbol rojinegro se utilizó la estructura `std::set` de la STL. Esta estructura mantiene los elementos ordenados y permite realizar operaciones de inserción y eliminación sobre el conjunto.

Los TODO #06, #07 y #08 corresponden a incluir la implementación utilizada, definir el tipo del árbol rojinegro y crear la estructura dentro del `main`. Al igual que en el AVL, se trabajó con datos de tipo `std::string` para almacenar las placas.

2.3.1. Inserción, eliminación y recorrido

Para la inserción se utiliza la operación `insert` del `std::set`. El valor retornado permite identificar si el elemento fue agregado correctamente. Para la eliminación se utiliza `erase`, que permite determinar si la clave se encontraba almacenada en la estructura.

El recorrido de los datos se realiza utilizando los iteradores del `std::set`. Como sus elementos permanecen ordenados, los valores se agregan a la lista directamente en orden ascendente.

La Figura 6 muestra las funciones utilizadas para estas operaciones.

El TODO #09 se completa pasando el árbol rojinegro a la función `LeerArbol`. De esta forma, las operaciones `add` y `del` de los archivos de prueba se aplican sobre esta estructura con el mismo mecanismo utilizado para el AVL.

Finalmente, el TODO #10 corresponde a almacenar el recorrido del árbol en una lista. Esta lista se utiliza posteriormente para comparar sus elementos con los obtenidos en las otras estructuras.

2.4. Montículo

La tercera estructura utilizada en el taller corresponde a un montículo máximo. A diferencia del AVL y del árbol rojinegro, el montículo no mantiene los elementos organizados como un árbol binario de búsqueda. Su propiedad principal consiste en conservar en cada nodo un valor mayor o igual que los valores de sus hijos, de manera que el elemento de mayor prioridad permanezca en la raíz.

Los TODO #11, #12 y #13 corresponden a incluir la implementación utilizada para el montículo, definir el tipo de estructura y crear la variable que será procesada junto con el AVL y el árbol rojinegro.

Para mantener una interfaz compatible con la función genérica `LeerArbol`, la implementación del montículo proporciona también los métodos `insert` y `erase`. Internamente los elementos se almacenan en una estructura lineal y se utilizan los mecanismos de montículo disponibles en la STL.

```cpp
#ifndef ARBOLRN_HXX
#define ARBOLRN_HXX

template <class T>
arbolRN<T>::arbolRN() {
}

template <class T>
bool arbolRN<T>::insert(T val) {

    //insert devuelve el iterador y si se pudo insertar
    std::pair<typename std::set<T>::iterator, bool> resultado;

    resultado = this->datos.insert(val);

    return resultado.second;
}

template <class T>
bool arbolRN<T>::erase(T val) {

    //erase devuelve cuantos elementos elimino
    return this->datos.erase(val) > 0;
}

template <class T>
void arbolRN<T>::inordenEnLista(std::list<T>& lista) {

    typename std::set<T>::iterator it;

    //set ya mantiene los datos ordenados
    for (it = this->datos.begin(); it != this->datos.end(); it++) {
        lista.push_back(*it);
    }
}

#endif
```

Figura 6 Implementación de inserción, eliminación y recorrido del árbol rojinegro.

2.4.1. Inserción en el montículo

Durante una inserción, el nuevo elemento se agrega al final de la estructura y posteriormente se reorganiza el montículo para conservar su propiedad de orden. Para esta operación se utiliza el mecanismo `push_heap` de la STL.

De esta forma, aunque el nuevo dato inicialmente se agrega en la siguiente posición disponible, puede desplazarse hacia niveles superiores hasta que se recupere la propiedad del montículo máximo.

La Figura 7 muestra las operaciones utilizadas para la inserción y eliminación de elementos.

Esta forma de inserción es diferente a la utilizada por los árboles de búsqueda. En el AVL y en el árbol rojinegro la ubicación del nuevo elemento depende de las comparaciones realizadas durante el recorrido del árbol. En el montículo, en cambio, primero se conserva la forma de árbol completo y luego se realizan los ajustes necesarios para recuperar la propiedad de prioridad.

2.4.2. Eliminación en el montículo

La eliminación también requiere un tratamiento diferente. Primero se localiza el elemento que se desea retirar dentro de la estructura. Una vez encontrado, se elimina

```
1   #ifndef MONTICULO_HXX
8   template <class T>
9   bool monticulo<T>::insert(T val) {
10
11      //el dato nuevo se agrega al final
12      this->datos.push_back(val);
13
14      //se reorganiza para mantener el monticulo
15      std::push_heap(this->datos.begin(), this->datos.end());
16
17      return true;
18  }
19
20  template <class T>
21  bool monticulo<T>::erase(T val) {
22
23      //se busca el dato que se quiere eliminar
24      typename std::deque<T>::iterator it =
25          std::find(this->datos.begin(), this->datos.end(), val);
26
27      if (it == this->datos.end())
28          return false;
29
30      //se elimina y se vuelve a organizar el monticulo
31      this->datos.erase(it);
32      std::make_heap(this->datos.begin(), this->datos.end());
33
34      return true;
35  }
36
37  template <class T>
38  void monticulo<T>::inordenEnLista(std::list<T>& lista) {
39
40      //se hace una copia para no modificar el monticulo
41      std::deque<T> copia = this->datos;
42
43      //la salida del monticulo debe quedar ordenada
44      std::sort(copia.begin(), copia.end());
45
```

Figura 7 Implementación de las operaciones de inserción y eliminación del montículo.

y posteriormente se reorganizan los elementos restantes para que el montículo vuelva a cumplir su propiedad.

En la implementación utilizada para el taller, después de retirar el elemento se emplea `make_heap` para reconstruir la organización del montículo. Esto permite que la estructura vuelva a quedar válida antes de continuar con la siguiente operación del archivo.

Esta diferencia es importante para el análisis posterior de rendimiento, ya que el comportamiento de una eliminación en el montículo no necesariamente es igual al de una inserción y tampoco sigue el mismo procedimiento utilizado en los árboles AVL y rojinegro.

2.4.3. Integración con la función de lectura

El TODO #14 corresponde a utilizar el archivo de entrada para llenar y modificar el montículo mediante la misma función genérica utilizada por las otras estructuras.

Al exponer las operaciones `insert` y `erase`, el montículo puede ser enviado directamente a `LeerArbol`. De esta forma, una misma secuencia de operaciones es ejecutada sobre las tres estructuras sin modificar el formato de los archivos de prueba.

La Figura 8 muestra la definición de la estructura y su conexión con la lectura del archivo.

```cpp
// TODO #13: Definir variable tipo Montículo.
THeap miMonticulo;

ReadStats statsAVL;
std::chrono::steady_clock::time_point t0AVL = std::chrono::steady_clock::now();

// TODO #04: Usar el archivo para llenar el árbol AVL a través del uso de la función genérica provista para ello.
LeerArbol(miArbolAVL, archivo, medirCadaOperacion, statsAVL);

std::chrono::steady_clock::time_point t1AVL = std::chrono::steady_clock::now();
double tiempoLecturaAVL = std::chrono::duration<double>(t1AVL - t0AVL).count();
statsAVL.secs_total = tiempoLecturaAVL;

ReadStats statsRN;
std::chrono::steady_clock::time_point t0RN = std::chrono::steady_clock::now();

// TODO #09: Usar el archivo para llenar el árbol rojinegro a través del uso de la función genérica provista para ello.
LeerArbol(miArbolRN, archivo, medirCadaOperacion, statsRN);
std::chrono::steady_clock::time_point t1RN = std::chrono::steady_clock::now();
double tiempoLecturaRN = std::chrono::duration<double>(t1RN - t0RN).count();
statsRN.secs_total = tiempoLecturaRN;

ReadStats statsHeap;
std::chrono::steady_clock::time_point t0Heap = std::chrono::steady_clock::now();

// TODO #14: Usar el archivo para llenar el montículo a través del uso de la función genérica provista para ello.
LeerArbol(miMonticulo, archivo, medirCadaOperacion, statsHeap);

std::chrono::steady_clock::time_point t1Heap = std::chrono::steady_clock::now();
double tiempoLecturaHeap = std::chrono::duration<double>(t1Heap - t0Heap).count();
statsHeap.secs_total = tiempoLecturaHeap;
```

Figura 8 Definición del montículo e integración con la función `LeerArbol`.

Los elementos utilizados por las tres estructuras provienen del mismo archivo, por lo que las diferencias observadas posteriormente no corresponden a datos de entrada distintos, sino al comportamiento de cada estructura al procesar las mismas operaciones.

2.4.4. Salida ordenada del montículo

El TODO #15 solicita almacenar los elementos del montículo en una lista para poder compararlos con los recorridos obtenidos en el AVL y el árbol rojinegro.

En este punto existe una diferencia importante. Un recorrido inorden sobre la representación del montículo no produce necesariamente los elementos en orden ascendente, ya que el montículo solamente garantiza la relación de prioridad entre cada padre y sus hijos.

Por esta razón, para realizar la comparación se genera una salida ordenada a partir de una copia de los datos del montículo. De esta manera se puede obtener una secuencia ascendente comparable con el recorrido inorden de las otras dos estructuras sin modificar el estado original del montículo.

La Figura 9 muestra la implementación utilizada para obtener esta secuencia.

La llamada correspondiente al TODO #15 se realiza después de procesar las operaciones del archivo y almacena el resultado en la lista destinada al montículo.

Gracias a esta salida ordenada es posible verificar que las tres estructuras mantienen el mismo conjunto de placas, incluso cuando su organización interna es diferente.

2.4.5. Resumen de la adaptación del montículo

Con los TODO #11 a #15 se integra la tercera estructura al programa. El montículo puede recibir las mismas operaciones de los archivos de prueba, insertar y

```
#ifndef MONTICULO_HXX
void monticulo<T>::inordenEnLista(std::list<T>& lista) {
    //la salida del monticulo debe quedar ordenada
    std::sort(copia.begin(), copia.end());

    typename std::deque<T>::iterator it;

    for (it = copia.begin(); it != copia.end(); it++) {
        lista.push_back(*it);
    }
}

#endif
```

Figura 9 Generación de la salida ordenada utilizada para comparar el montículo con los árboles.

```
// TODO #15: Llamar la función que genera el recorrido en inorden del montículo y lo guarda en una lista dada como parámetro.
miMonticulo.inordenEnLista(inordenHeap);


std::size_t sizeAVL  = inordenAVL.size();
std::size_t sizeRN   = inordenRN.size();
std::size_t sizeHeap = inordenHeap.size();

std::size_t minSize = std::min({sizeAVL, sizeRN, sizeHeap});

std::cout << "\n----------------- Resumen de tamaños (inorden) -----------------\n";
std::cout << " AVL               : " << sizeAVL  << " elementos\n";
std::cout << " Árbol Rojo-Negro  : " << sizeRN   << " elementos\n";
std::cout << " Heap              : " << sizeHeap << " elementos\n";

if (sizeAVL == sizeRN && sizeRN == sizeHeap) {
  std::cout << " ==> Las tres estructuras tienen la misma cantidad de elementos.\n";
} else {
  std::cout << " ==> La estructura con menos elementos tiene " << minSize << " elemento(s).\n";
}
```

Figura 10 Llamada utilizada para almacenar la salida del montículo en la lista de comparación.

eliminar elementos y producir una salida que posteriormente puede compararse con las obtenidas por el AVL y el árbol rojinegro.

La principal diferencia con las otras estructuras se encuentra en su forma de organización. Mientras que el AVL y el árbol rojinegro mantienen un orden de búsqueda entre los subárboles izquierdo y derecho, el montículo conserva una relación de prioridad. Por este motivo, tanto los ajustes después de las operaciones como la forma de generar una secuencia ordenada requieren un tratamiento diferente.

2.5. Comparación de los resultados

Después de procesar el mismo archivo sobre el AVL, el árbol rojinegro y el montículo, el programa genera una estructura lineal para cada uno de los resultados. En el caso del AVL y del árbol rojinegro se utiliza el recorrido inorden, mientras que para el montículo se emplea una salida ordenada que permita realizar una comparación equivalente.

El objetivo de los TODO #16, #17 y #18 es recorrer estas tres listas y verificar que los elementos almacenados coincidan tanto en cantidad como en posición.

2.5.1. Creación de iteradores

El TODO #16 corresponde a crear los iteradores necesarios para recorrer las listas obtenidas de cada estructura.

Cada iterador comienza en el primer elemento de su respectiva lista y avanza simultáneamente con los demás. De esta manera se puede comparar la posición actual de las tres secuencias durante una misma iteración.

La Figura 11 muestra la creación de los iteradores y la lógica utilizada durante la comparación.

```
// TODO #16: Crear iteradores para recorrer cada una de las estructuras lineales
TList::iterator itAVL  = inordenAVL.begin();
TList::iterator itRN   = inordenRN.begin();
TList::iterator itHeap = inordenHeap.begin();

// TODO #17: Recorrer las estructuras lineales y comparar elemento a elemento la igualdad o desigualdad
bool todosIguales = true;

for (std::size_t i = 0; i < minSize; ++i, ++itAVL, ++itRN, ++itHeap) {

  if (!(*itAVL == *itRN && *itRN == *itHeap)) {

    std::cout << " Diferencia en la posicion " << i
              << " | AVL: " << *itAVL
              << " | RN: "  << *itRN
              << " | Heap: "<< *itHeap << "\n";

    todosIguales = false;
  }
}
```

Figura 11 Creación de iteradores y comparación de las listas obtenidas de las tres estructuras.

Antes de comparar los valores también es necesario tener en cuenta el tamaño de las listas. Si una estructura contiene una cantidad distinta de elementos, no es posible considerar que las tres coinciden completamente.

2.5.2. Comparación elemento por elemento

El TODO #17 recorre simultáneamente las listas y compara los valores almacenados en la misma posición.

La comparación se realiza sobre las secuencias ordenadas y no sobre la forma interna de cada estructura. Esto es importante porque un AVL, un árbol rojinegro y un montículo pueden organizar físicamente los mismos datos de formas diferentes.

Por ejemplo, dos estructuras pueden tener raíces distintas y aun así contener exactamente el mismo conjunto de placas. Por esta razón, para esta verificación se utilizan las listas generadas después de finalizar todas las operaciones del archivo de prueba.

Durante el recorrido, si se encuentra una posición en la que alguno de los valores es diferente, se registra que las estructuras no coinciden en su totalidad.

Si los tres valores son iguales, los iteradores continúan avanzando hasta alcanzar el final de las listas.

Esta comprobación permite revisar de forma directa que una inserción, eliminación o reorganización interna de alguna estructura no haya producido una diferencia en el conjunto final de datos.

2.5.3. Resultado de la comparación

Finalmente, el TODO #18 informa el resultado obtenido después de revisar las listas completas.

La Figura 12 muestra la parte del programa encargada de presentar este resultado.

```
// TODO #18: Informar si los árboles coinciden en la totalidad de los elementos teniendo en cuenta su posición
if (todosIguales && sizeAVL == sizeRN && sizeRN == sizeHeap) {

  std::cout << " ==> Coincidencia total: los tres recorridos inorden son idénticos.\n";

} else if (todosIguales) {

  std::cout << " ==> Coincidencia parcial: igualdad en posiciones comparadas, pero tamaños distintos.\n";

} else {

  std::cout << " ==> No hay coincidencia total entre las tres estructuras.\n";
}

  return (0);
}

template <class TArbol>
bool LeerArbol(TArbol &arbol, const std::string &nomArch, bool medirCadaOperacion, ReadStats &stats)
{
  std::ifstream entrada(nomArch.c_str());
  if (!entrada) return false;

  std::string codigo, valor;
  using clock_t = std::chrono::steady_clock;

  while (entrada >> codigo >> valor)
  {
    std::chrono::steady_clock::time_point t0 = clock_t::now();

    if (codigo == "add") {
      arbol.insert(valor);
      std::chrono::steady_clock::time_point t1 = clock_t::now();
      if (medirCadaOperacion) {
        double secs = std::chrono::duration<double>(t1 - t0).count();
        stats.secs_add += secs;
      }
```

Figura 12 Resultado final de la comparación entre AVL, árbol rojinegro y montículo.

Cuando las listas tienen el mismo tamaño y todos sus elementos coinciden en la misma posición, el programa informa que existe coincidencia total entre las tres estructuras.

Esta comparación fue especialmente útil durante las pruebas, ya que permitió verificar que las diferencias internas entre AVL, rojinegro y montículo no afectaran el resultado final de las operaciones realizadas.

2.5.4. Verificación con los archivos de prueba

La prueba con el archivo `01_solo_agregar_10.txt` produjo la misma secuencia ordenada para las tres estructuras:

```
BBB060, DDD125, FFF190, GGG250, HHH310,
JJJ375, MMM500, PPP625, TTT750, WWW875
```

De forma similar, después de procesar el archivo `07_agregar_10_quitar_5.txt`, las tres estructuras conservaron los mismos cinco elementos:

```
FFF190, HHH310, JJJ375, PPP625, WWW875
```

En ambos casos el programa informó coincidencia total. Esto permitió continuar con el desarrollo gráfico y posteriormente con las mediciones de tiempo, teniendo primero una comprobación funcional de que las tres estructuras estaban procesando correctamente los mismos datos.

3. Punto 7: desarrollo gráfico de las inserciones

Para este punto se utilizó el archivo `01_solo_agregar_10.txt`. Las diez claves se insertan en el siguiente orden:

`MMM500, GGG250, TTT750, DDD125, JJJ375,`
`PPP625, WWW875, BBB060, FFF190, HHH310`

Las tres estructuras parten vacías y reciben exactamente la misma secuencia de datos. Sin embargo, cada una realiza ajustes distintos debido a sus propiedades internas. Primero se desarrolla el proceso correspondiente al árbol AVL.

3.1. Inserciones en el árbol AVL

En el árbol AVL cada clave se inserta siguiendo inicialmente las reglas de un árbol binario de búsqueda. Los valores menores se ubican en el subárbol izquierdo y los mayores en el subárbol derecho.

Después de cada inserción se revisa el equilibrio de los nodos involucrados. Para que el árbol conserve la propiedad AVL, la diferencia entre las alturas de los subárboles izquierdo y derecho de cada nodo debe mantenerse entre -1 y 1.

3.1.1. Inserción de `MMM500`

La primera clave se inserta cuando el árbol todavía está vacío. Por esta razón, `MMM500` pasa directamente a ser la raíz.

En este momento el árbol contiene un único nodo, por lo que no existe ningún desequilibrio que deba corregirse.

3.1.2. Inserción de `GGG250`

La siguiente clave es `GGG250`. Al compararla con `MMM500`, se determina que es menor, por lo que se ubica como hijo izquierdo de la raíz.

La diferencia de alturas de la raíz continúa dentro del rango permitido, por lo que no se requiere ninguna rotación.

3.1.3. Inserción de `TTT750`

La clave `TTT750` es mayor que `MMM500`, por lo que se ubica como hijo derecho de la raíz.

Con esta inserción la raíz queda con un hijo a cada lado y los dos subárboles tienen la misma altura.

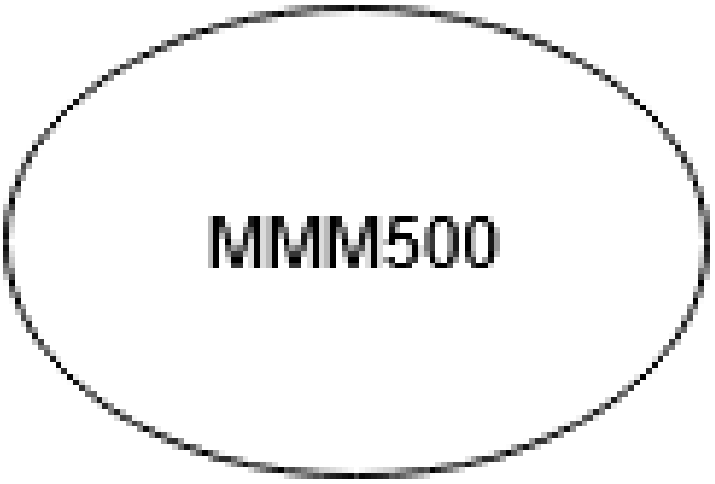


Figura 13 Árbol AVL después de insertar `MMM500`.



Figura 14 Árbol AVL después de insertar `GGG250`.

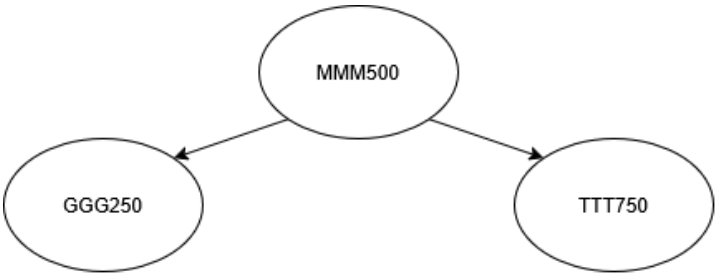


Figura 15 Árbol AVL después de insertar `TTT750`.

3.1.4. Inserción de `DDD125`

Para insertar `DDD125`, primero se compara con `MMM500`. Como es menor, se continúa por el subárbol izquierdo. Después se compara con `GGG250` y nuevamente resulta menor, por lo que queda como hijo izquierdo de este nodo.

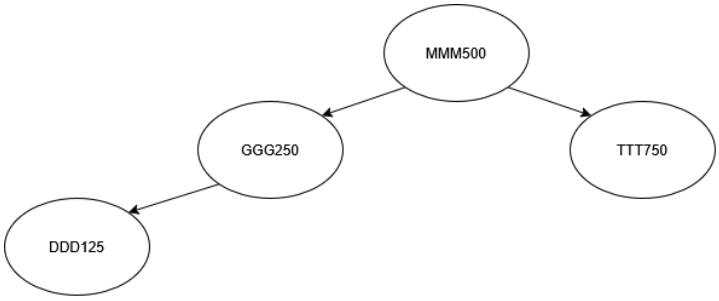


Figura 16 Árbol AVL después de insertar `DDD125`.

Aunque aumenta la altura del lado izquierdo, el factor de equilibrio continúa dentro de los valores permitidos y todavía no es necesario realizar rotaciones.

3.1.5. Inserciones de `JJJ375` y `PPP625`

La clave `JJJ375` es menor que `MMM500`, pero mayor que `GGG250`, por lo que se ubica como hijo derecho de `GGG250`.

Después se inserta `PPP625`. Esta clave es mayor que `MMM500` y menor que `TTT750`, por lo que queda como hijo izquierdo de `TTT750`.

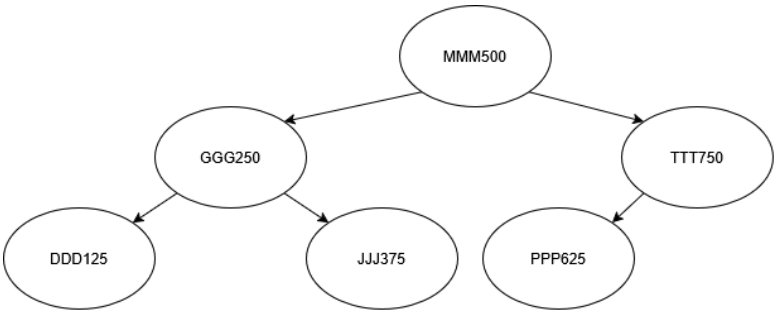


Figura 17 Árbol AVL después de insertar `JJJ375` y `PPP625`.

Después de estas inserciones se vuelve a comprobar el balance de los nodos afectados. Ninguno presenta una diferencia de alturas por fuera del intervalo permitido.

3.1.6. Inserción de `WWW875`

La clave `WWW875` es mayor que `MMM500` y también mayor que `TTT750`. Por esta razón se agrega como hijo derecho de `TTT750`.

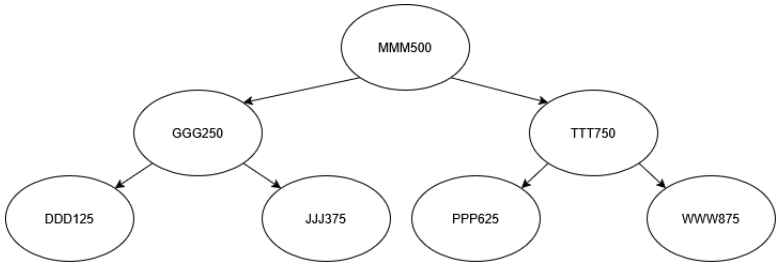


Figura 18 Árbol AVL después de insertar `WWW875`.

El nodo `TTT750` queda con un hijo a cada lado y el árbol continúa equilibrado.

3.1.7. Inserción de `BBB060`

Para insertar `BBB060` se recorre el lado izquierdo del árbol. La clave es menor que `MMM500`, menor que `GGG250` y también menor que `DDD125`. Finalmente se ubica como hijo izquierdo de `DDD125`.

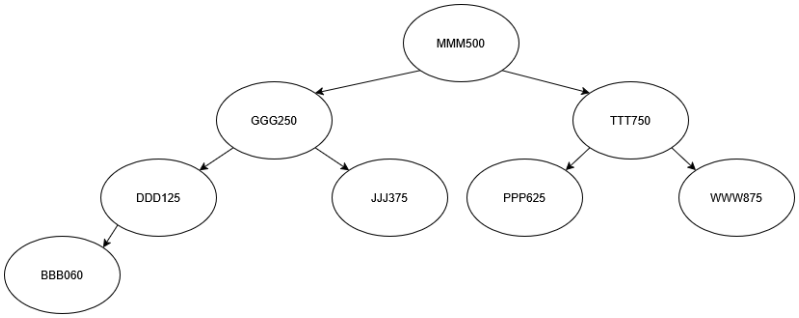


Figura 19 Árbol AVL después de insertar `BBB060`.

El subárbol izquierdo aumenta nuevamente su altura, pero los factores de equilibrio de los ancestros siguen siendo válidos.

3.1.8. Inserción de `FFF190`

La clave `FFF190` es menor que `MMM500` y `GGG250`, pero mayor que `DDD125`. Por lo tanto, se ubica como hijo derecho de `DDD125`.

Con los hijos `BBB060` y `FFF190`, el nodo `DDD125` queda equilibrado y tampoco se requiere realizar una rotación.

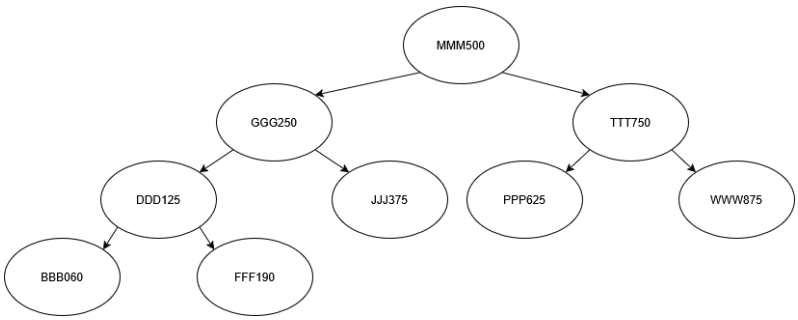


Figura 20 Árbol AVL después de insertar `FFF190`.

3.1.9. Inserción de `HHH310`

La última clave del archivo es `HHH310`. Primero se dirige hacia el subárbol izquierdo de `MMM500`. Después, al ser mayor que `GGG250` pero menor que `JJJ375`, queda ubicada como hijo izquierdo de `JJJ375`.

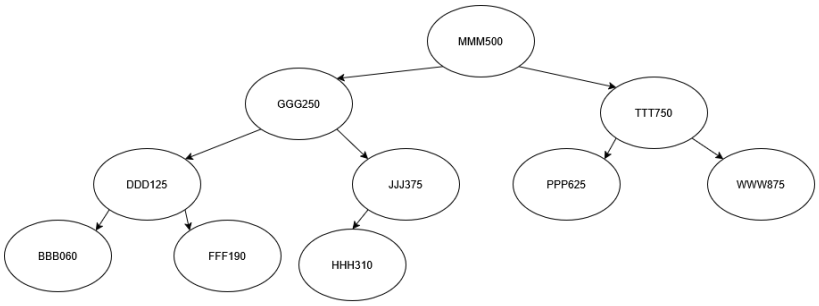


Figura 21 Árbol AVL final después de las diez inserciones.

Después de insertar las diez claves, todos los nodos mantienen factores de equilibrio entre -1 y 1. Por esta razón, para esta secuencia particular de inserciones no fue necesario realizar ninguna rotación.

El recorrido inorden del árbol final produce la siguiente secuencia:

```
BBB060, DDD125, FFF190, GGG250, HHH310,
JJJ375, MMM500, PPP625, TTT750, WWW875
```

Esta secuencia coincide con el resultado obtenido posteriormente durante la ejecución del programa.

3.2. Inserciones en el árbol rojinegro

Para el desarrollo gráfico del árbol rojinegro se utiliza la misma secuencia de diez claves aplicada al AVL. En este caso, cada nodo nuevo se inserta inicialmente de color rojo, excepto cuando corresponde a la raíz, que debe permanecer negra.

Después de cada inserción se revisan las propiedades del árbol rojinegro. Si el padre del nuevo nodo también es rojo, se analiza el color del tío para determinar si se debe realizar un recoloreo o una rotación.

3.2.1. Inserción de `MMM500`

La primera clave insertada es `MMM500`. Como el árbol se encuentra vacío, este nodo se convierte en la raíz y se establece de color negro.



Figura 22 Árbol rojinegro después de insertar `MMM500`.

No se requiere ningún ajuste adicional, ya que la raíz cumple directamente con las propiedades del árbol rojinegro.

3.2.2. Inserciones de `GGG250` y `TTT750`

La clave `GGG250` es menor que `MMM500`, por lo que se ubica como hijo izquierdo. La clave `TTT750` es mayor y se ubica como hijo derecho.

Ambos nodos se insertan inicialmente de color rojo.

Como los dos nodos tienen como padre a `MMM500`, que es negro, no se presenta ninguna violación de las propiedades del árbol.

3.2.3. Inserción de `DDD125` y primer recoloreo

La clave `DDD125` es menor que `MMM500` y que `GGG250`, por lo que se inserta como hijo izquierdo de `GGG250`.

El nuevo nodo es rojo y su padre, `GGG250`, también es rojo. En este caso el tío del nodo corresponde a `TTT750`, que también es rojo.

Por esta razón se realiza un recoloreo: `GGG250` y `TTT750` pasan a negro, mientras que `MMM500` cambia temporalmente a rojo. Como `MMM500` es la raíz, finalmente vuelve a establecerse de color negro.

Este ajuste permite recuperar las propiedades del árbol sin necesidad de realizar rotaciones.



Figura 23 Árbol rojinegro después de insertar `GGG250` y `TTT750`.

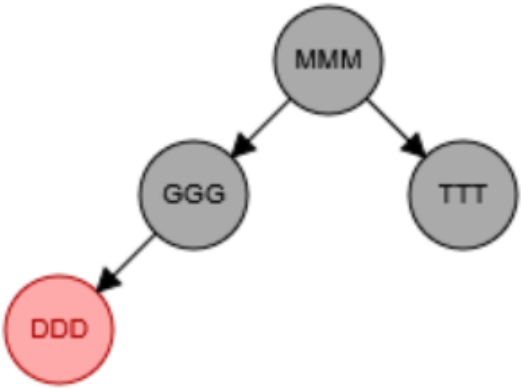


Figura 24 Árbol rojinegro después de insertar `DDD125` y realizar el recoloreo.

3.2.4. Inserciones de `JJJ375`, `PPP625` y `WWW875`

La clave `JJJ375` se ubica como hijo derecho de `GGG250`. La clave `PPP625` se inserta como hijo izquierdo de `TTT750` y `WWW875` como hijo derecho de este mismo nodo.

Los tres nodos se agregan de color rojo.

En los tres casos el padre correspondiente es negro, por lo que no se generan pares de nodos rojos consecutivos y no se requieren ajustes adicionales.

3.2.5. Inserción de `BBB060` y segundo recoloreo

La clave `BBB060` es menor que `MMM500`, `GGG250` y `DDD125`, por lo que se ubica como hijo izquierdo de `DDD125`.

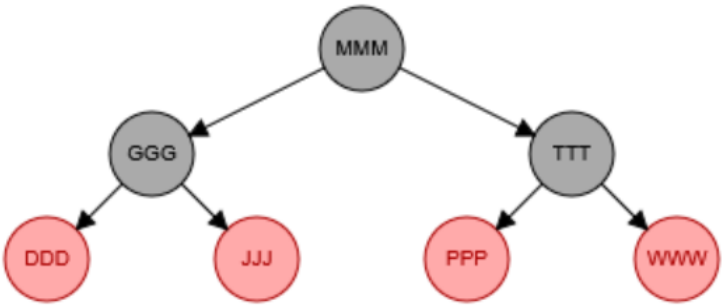


Figura 25 Árbol rojinegro después de insertar `JJJ375`, `PPP625` y `WWW875`.

El nuevo nodo es rojo y su padre también es rojo. El tío del nuevo nodo es `JJJ375`, que igualmente es rojo. Se presenta entonces el mismo tipo de situación observada anteriormente.

Se recolorean `DDD125` y `JJJ375` a negro, mientras que `GGG250` cambia a rojo. Como el padre de `GGG250`, `MMM500`, es negro, no es necesario continuar con más ajustes.

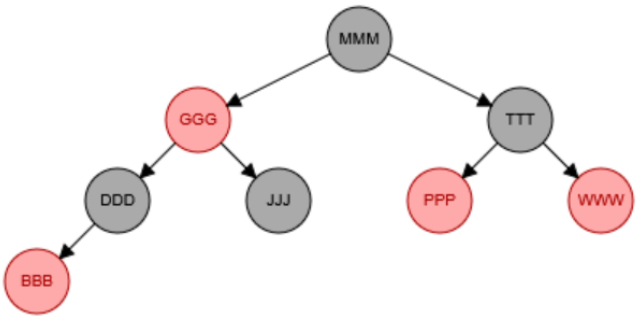


Figura 26 Árbol rojinegro después de insertar `BBB060` y realizar el segundo recoloreo.

3.2.6. Inserciones de `FFF190` y `HHH310`

Las dos últimas claves de la secuencia se insertan sin producir nuevas violaciones.

`FFF190` se ubica como hijo derecho de `DDD125`. Como `DDD125` es negro, no se requiere ningún ajuste.

Por su parte, `HHH310` se ubica como hijo izquierdo de `JJJ375`. Este nodo también tiene un padre negro, por lo que la estructura conserva las propiedades del árbol rojinegro.

Durante esta secuencia se realizaron dos recoloreos, uno después de insertar `DDD125` y otro después de insertar `BBB060`. No fue necesario realizar rotaciones.

El recorrido inorden del árbol final produce:

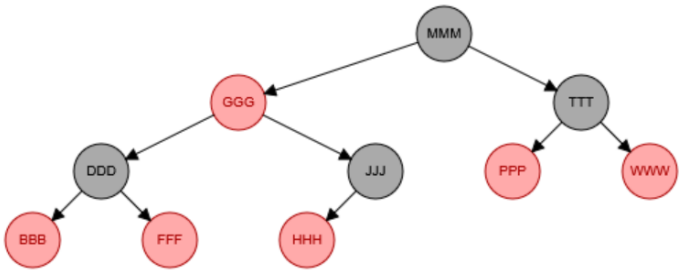


Figura 27 Árbol rojinegro final después de las diez inserciones.

```
BBB060, DDD125, FFF190, GGG250, HHH310,
JJJ375, MMM500, PPP625, TTT750, WWW875
```

La secuencia coincide con la obtenida en el AVL y con el resultado observado durante la ejecución del programa.

3.3. Inserciones en el montículo máximo

Para el montículo máximo se utiliza la misma secuencia de diez claves aplicada a los árboles AVL y rojinegro. En este caso, cada nuevo elemento se ubica primero en la siguiente posición disponible del árbol completo.

Después de insertar el nuevo elemento se compara con su padre. Si el valor insertado es mayor, ambos se intercambian. Este proceso puede repetirse hasta que se recupere la propiedad del montículo máximo, en la cual cada padre es mayor o igual que sus hijos.

3.3.1. Inserción de `MMM500`

La primera clave se inserta en un montículo vacío, por lo que pasa directamente a ser la raíz.

No se requiere ningún ajuste porque todavía no existen otros elementos con los cuales realizar comparaciones.

3.3.2. Inserción de `GGG250`

La clave `GGG250` se ubica en la siguiente posición disponible, como hijo izquierdo de `MMM500`.

Como `GGG250` es menor que su padre, conserva su posición y no se realiza ningún intercambio.

3.3.3. Inserción de `TTT750`

La clave `TTT750` ocupa la siguiente posición disponible, como hijo derecho de `MMM500`.

Al comparar el nuevo nodo con su padre se observa que `TTT750` es mayor que `MMM500`. Por esta razón se realiza un intercambio entre ambos elementos.



Figura 28 Montículo después de insertar `MMM500`.



Figura 29 Montículo después de insertar `GGG250`.

Después del intercambio, `TTT750` queda como raíz y la propiedad del montículo vuelve a cumplirse.

3.3.4. Inserción de `DDD125`

La clave `DDD125` se agrega en la siguiente posición disponible, como hijo izquierdo de `GGG250`.

Como `DDD125` es menor que su padre, no es necesario modificar su posición.

3.3.5. Inserción de `JJJ375`

La clave `JJJ375` se ubica como hijo derecho de `GGG250`.

En este caso `JJJ375` es mayor que `GGG250`, por lo que ambos elementos deben intercambiarse.

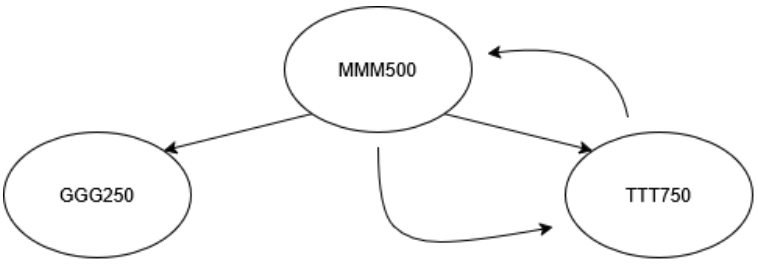


Figura 30 Montículo después de ubicar `TTT750` como hijo derecho de `MMM500`.

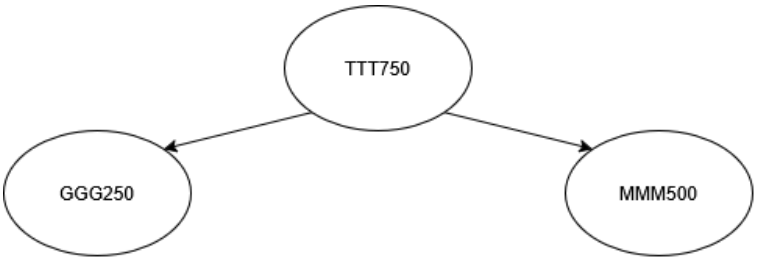


Figura 31 Montículo después del intercambio entre `TTT750` y `MMM500`.

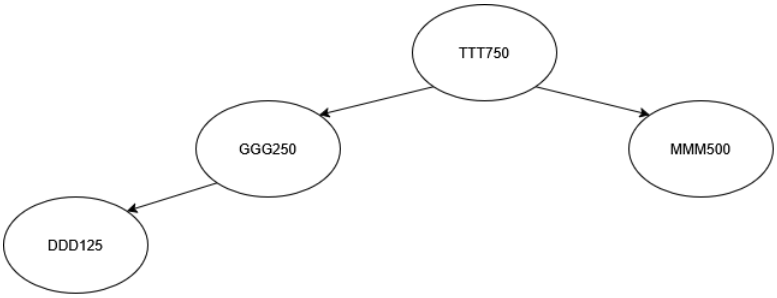


Figura 32 Montículo después de insertar `DDD125`.

Después del intercambio, `JJJ375` queda por debajo de `TTT750`. Como continúa siendo menor que su nuevo padre, el ajuste termina en ese punto.

3.3.6. Inserción de `PPP625`

La siguiente clave, `PPP625`, se ubica como hijo izquierdo de `MMM500`.

Al comparar ambos valores se observa que `PPP625` es mayor que `MMM500`. Por esta razón se realiza un intercambio.

Después de subir un nivel, `PPP625` queda debajo de `TTT750`. Como sigue siendo menor que este nodo, no se requieren más intercambios.

3.3.7. Inserción de `WWW875`

La clave `WWW875` se ubica inicialmente como hijo derecho de `PPP625`.

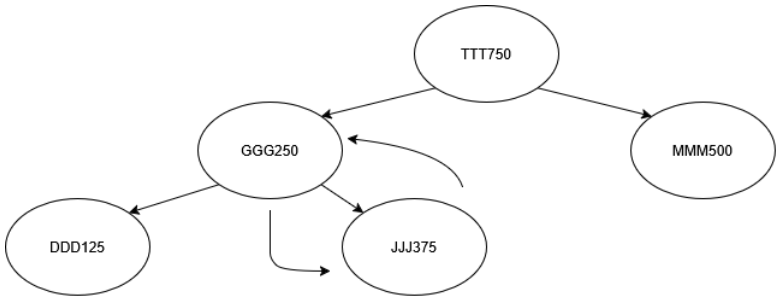


Figura 33 Montículo después de ubicar `JJJ375` como hijo derecho de `GGG250`.

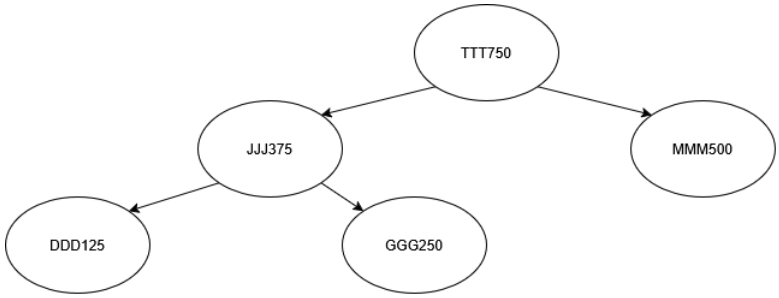


Figura 34 Montículo después del intercambio entre `JJJ375` y `GGG250`.

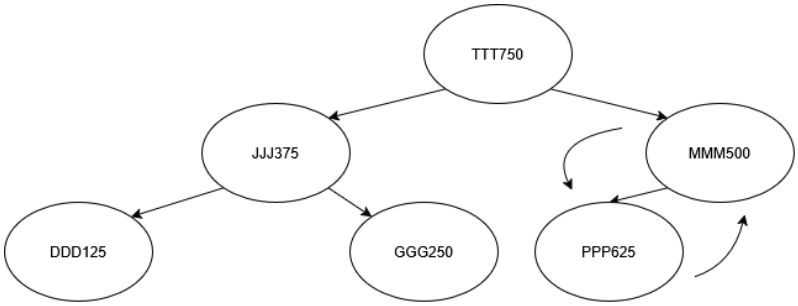


Figura 35 Montículo después de ubicar `PPP625` como hijo izquierdo de `MMM500`.

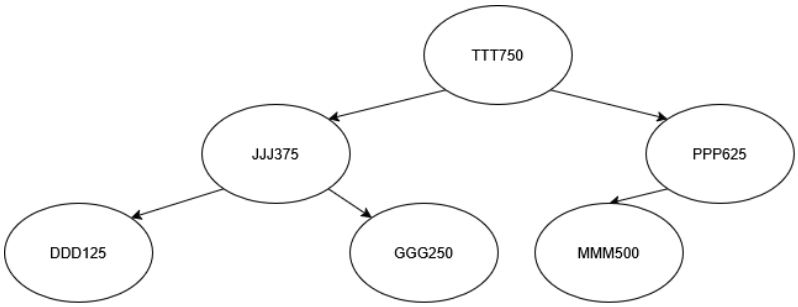


Figura 36 Montículo después del intercambio entre `PPP625` y `MMM500`.

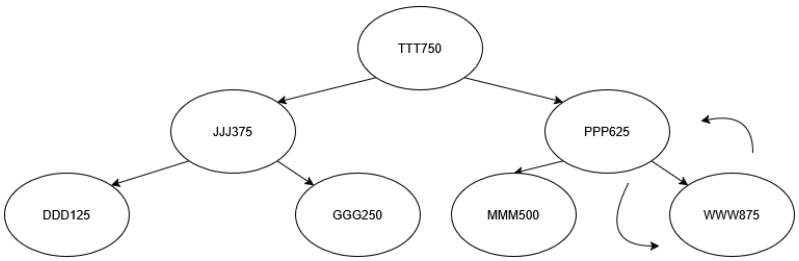


Figura 37 Montículo después de ubicar `WWW875` como hijo derecho de `PPP625`.

Como `WWW875` es mayor que `PPP625`, se intercambian.

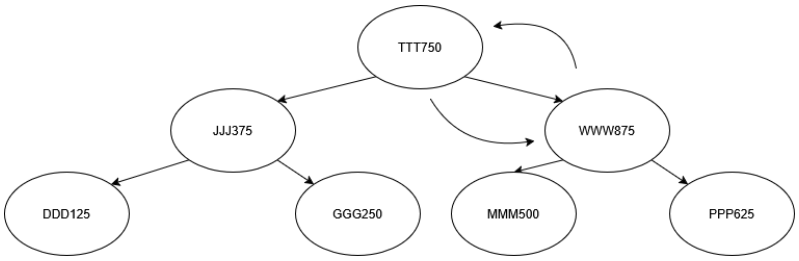


Figura 38 Montículo después del intercambio entre `WWW875` y `PPP625`.

Sin embargo, después de este primer intercambio `WWW875` queda como hijo de `TTT750`. Como `WWW875` también es mayor que este nuevo padre, se realiza un segundo intercambio.

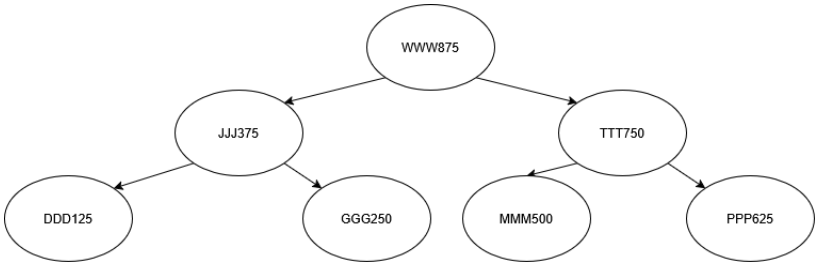


Figura 39 Montículo después del intercambio entre `WWW875` y `TTT750`.

Con este ajuste, `WWW875` pasa a ser la raíz del montículo.

3.3.8. Inserción de `BBB060`

La siguiente clave es `BBB060`. Se ubica en la primera posición libre del siguiente nivel, como hijo izquierdo de `DDD125`.

Como el nuevo valor es menor que su padre, no es necesario realizar ningún intercambio.

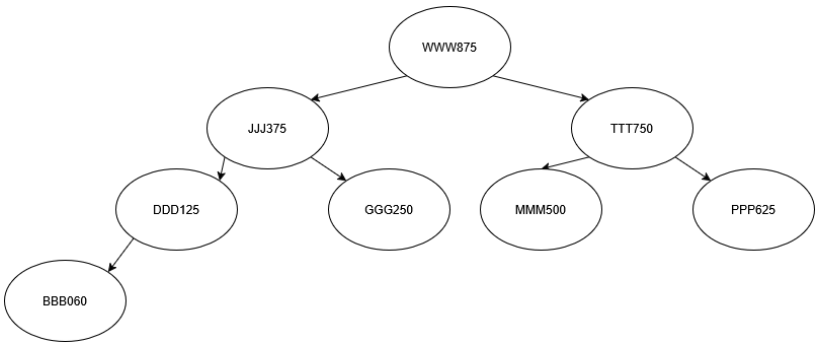


Figura 40 Montículo después de insertar `BBB060`.

3.3.9. Inserción de `FFF190`

La clave `FFF190` se ubica como hijo derecho de `DDD125`.

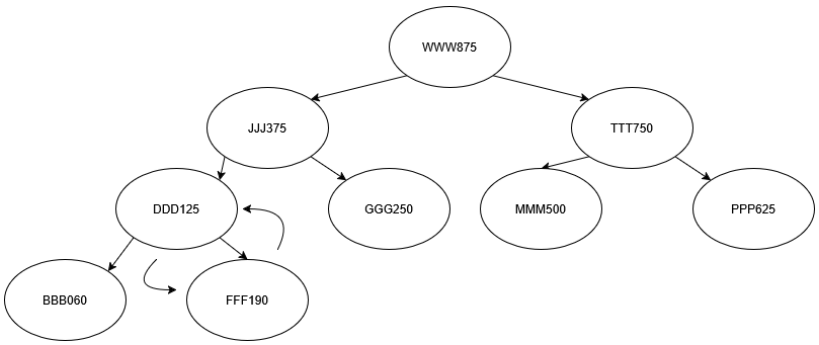


Figura 41 Montículo después de ubicar `FFF190` como hijo derecho de `DDD125`.

Como `FFF190` es mayor que `DDD125`, ambos valores se intercambian.

Después del intercambio, `FFF190` queda debajo de `JJJ375`. Como es menor que su nuevo padre, el proceso termina.

3.3.10. Inserción de `HHH310`

La última clave de la secuencia es `HHH310`. Esta se ubica inicialmente como hijo izquierdo de `GGG250`.

Al comparar los valores se observa que `HHH310` es mayor que `GGG250`. Por esta razón se intercambian. Después del ajuste, `HHH310` queda debajo de `JJJ375`, que continúa siendo mayor, por lo que no se requieren más cambios.

Después de las diez inserciones se obtiene un montículo máximo cuya raíz es `WWW875`. Durante la construcción se realizaron varios intercambios para mantener la propiedad de prioridad de la estructura.

La representación final del montículo es:

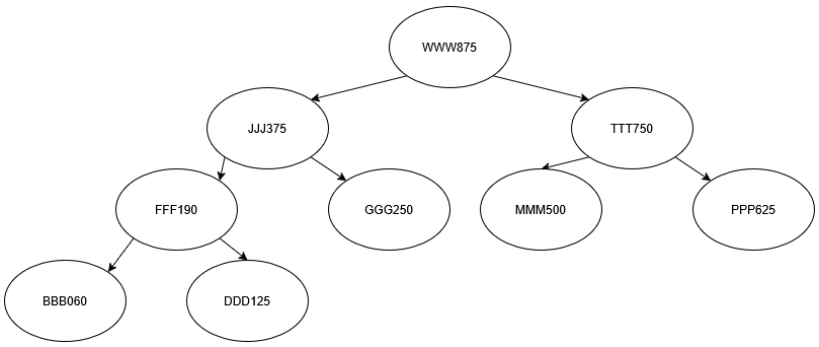


Figura 42 Montículo después del intercambio entre `FFF190` y `DDD125`.

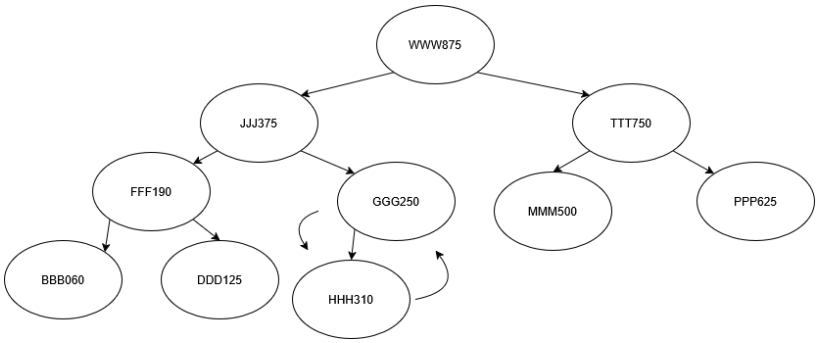


Figura 43 Montículo después de ubicar `HHH310` como hijo izquierdo de `GGG250`.

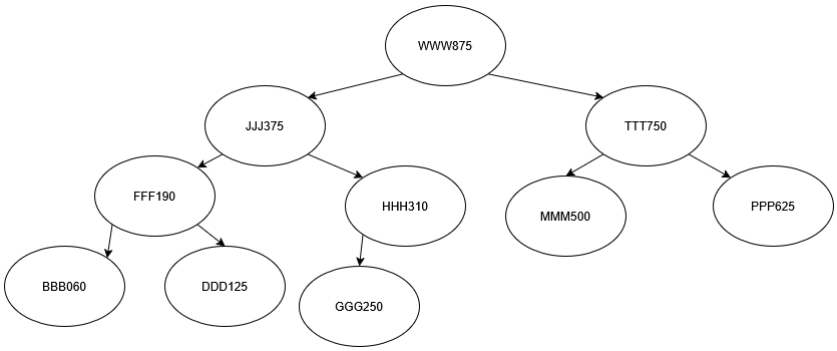


Figura 44 Montículo máximo final después de las diez inserciones.

```
[WWW875, JJJ375, TTT750, FFF190, HHH310,
MMM500, PPP625, BBB060, DDD125, GGG250]
```

A diferencia de los árboles AVL y rojinegro, un recorrido inorden del montículo no produce necesariamente una secuencia ascendente. Por esta razón, para la comparación con las otras dos estructuras se utiliza una salida ordenada.

La secuencia ascendente esperada es:

```
BBB060, DDD125, FFF190, GGG250, HHH310,
JJJ375, MMM500, PPP625, TTT750, WWW875
```

Esta secuencia coincide con las obtenidas mediante los recorridos inorden del AVL y del árbol rojinegro.

4. Punto 8: desarrollo gráfico de las eliminaciones

Para este punto se parte de las estructuras obtenidas después de las diez inserciones del punto anterior. Las claves se eliminan en el siguiente orden:

```
BBB060, DDD125, GGG250, TTT750, MMM500
```

En los árboles de búsqueda se presentan los casos habituales de eliminación: retirar una hoja, reemplazar un nodo con un único hijo y reemplazar un nodo con dos hijos por su predecesor.

A partir de cada eliminación se revisan nuevamente las propiedades particulares de cada estructura.

4.1. Eliminaciones en el árbol AVL

El proceso comienza con el árbol AVL obtenido al finalizar las diez inserciones del Punto 7.

Después de cada eliminación se revisa el equilibrio de los nodos afectados. Si algún factor de equilibrio queda fuera del intervalo permitido, se realizan las rotaciones necesarias para recuperar la propiedad AVL.

4.1.1. Eliminación de `BBB060`

La primera clave que se elimina es `BBB060`. Este nodo corresponde a una hoja, por lo que puede retirarse directamente sin tener que reemplazarlo por otro elemento.

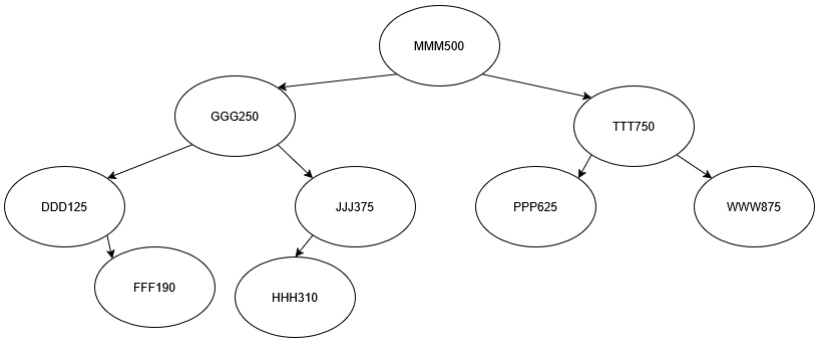


Figura 45 Árbol AVL después de eliminar `BBB060`.

Después de retirar el nodo, el árbol mantiene factores de equilibrio válidos y no se requiere ninguna rotación.

4.1.2. Eliminación de `DDD125`

La siguiente clave es `DDD125`. En este momento el nodo tiene un único hijo, `FFF190`.

Por esta razón, `FFF190` pasa a ocupar directamente la posición que tenía `DDD125` como hijo izquierdo de `GGG250`.

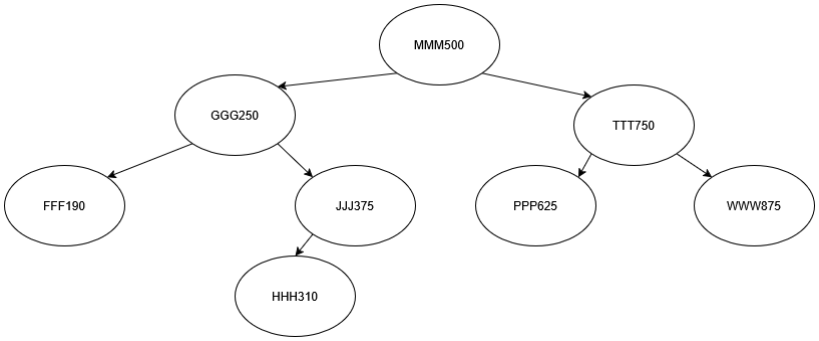


Figura 46 Árbol AVL después de eliminar `DDD125`.

Después del reemplazo se revisan nuevamente las alturas de los subárboles. El árbol continúa equilibrado y no necesita ajustes adicionales.

4.1.3. Eliminación de `GGG250`

La clave `GGG250` tiene dos hijos. Para realizar la eliminación se utiliza su predecesor, es decir, la clave mayor del subárbol izquierdo.

En este caso el predecesor corresponde a `FFF190`, que reemplaza a `GGG250`.

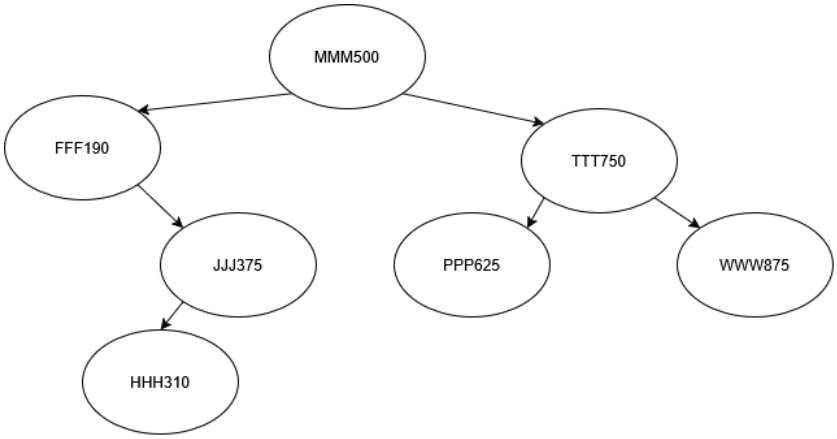


Figura 47 Árbol AVL después de reemplazar `GGG250` por su predecesor `FFF190`.

Después de realizar el reemplazo se presenta un desequilibrio en este subárbol. El lado derecho tiene una altura mayor y, además, el hijo derecho presenta una inclinación hacia la izquierda.

Esta configuración corresponde a un caso derecha–izquierda, por lo que es necesario realizar una rotación doble.

4.1.4. Primera rotación de la corrección

El primer paso consiste en realizar una rotación a la derecha sobre `JJJ375`.

Con este movimiento, `HHH310` asciende dentro del subárbol y `JJJ375` queda como su hijo derecho.

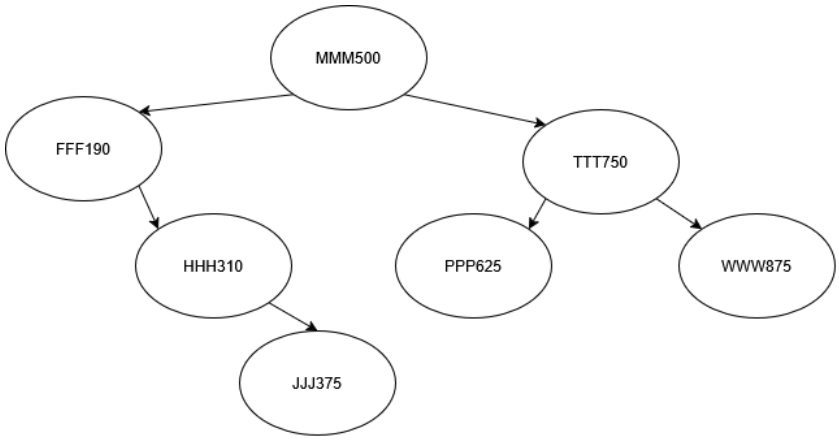


Figura 48 Árbol AVL después de la rotación a la derecha sobre `JJJ375`.

Esta primera rotación prepara la estructura para completar la corrección del desequilibrio.

4.1.5. Segunda rotación de la corrección

A continuación se realiza una rotación a la izquierda sobre `FFF190`.

Después de este movimiento, `HHH310` queda como raíz del subárbol, con `FFF190` como hijo izquierdo y `JJJ375` como hijo derecho.

Con esta segunda rotación se completa la corrección derecha–izquierda y el subárbol recupera su equilibrio.

4.1.6. Eliminación de `TTT750`

La siguiente clave que se elimina es `TTT750`. Este nodo tiene dos hijos, por lo que nuevamente se utiliza su predecesor.

El predecesor corresponde a `PPP625`, que pasa a ocupar la posición de `TTT750`. El nodo `WWW875` permanece como hijo derecho.

Después del reemplazo, las alturas de los dos lados principales continúan dentro del rango permitido, por lo que no se requieren nuevas rotaciones.

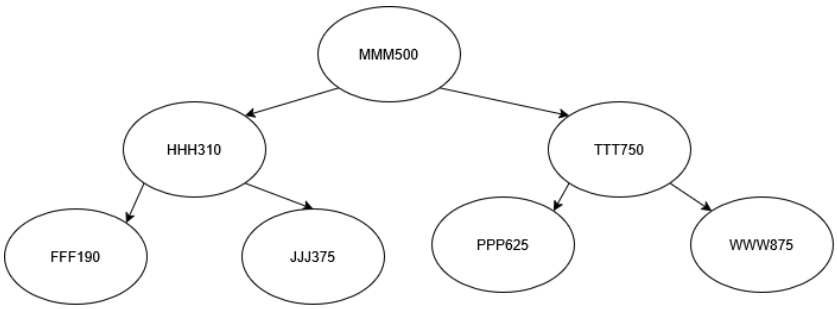


Figura 49 Árbol AVL después de la rotación a la izquierda sobre `FFF190`.

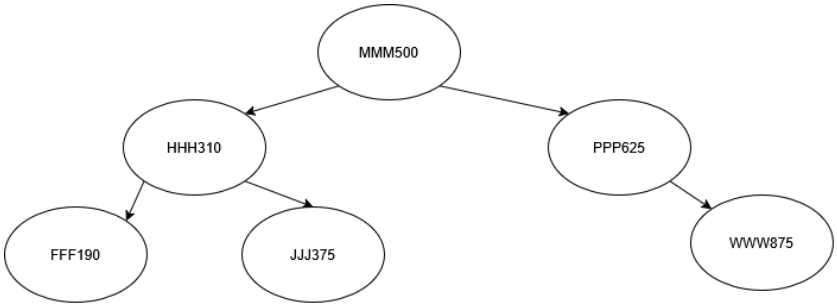


Figura 50 Árbol AVL después de eliminar `TTT750`.

4.1.7. Eliminación de `MMM500`

La última clave eliminada es `MMM500`, que corresponde a la raíz del árbol.

Como tiene dos hijos, se utiliza nuevamente el predecesor. En este caso, `JJJ375` pasa a ocupar la raíz.

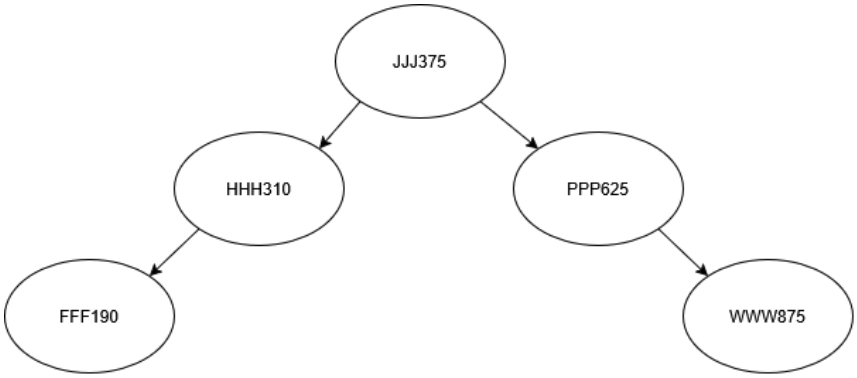


Figura 51 Árbol AVL final después de las cinco eliminaciones.

Después de las cinco eliminaciones quedan los siguientes elementos:

FFF190, HHH310, JJJ375, PPP625, WWW875

El árbol final conserva la propiedad AVL. Durante este proceso fue necesario realizar una única rotación doble derecha–izquierda, originada después de la eliminación de GGG250.

El recorrido inorden del árbol final produce precisamente la secuencia anterior, que coincide con el resultado obtenido durante la ejecución del programa.

4.2. Eliminaciones en el árbol rojinegro

Para el árbol rojinegro se parte de la estructura obtenida después de las diez inserciones del Punto 7. Al eliminar elementos no solo debe conservarse el orden propio de un árbol binario de búsqueda, sino también las propiedades de color de la estructura.

Dependiendo del color del nodo eliminado y de sus hijos, puede ser necesario realizar recoloreos o rotaciones para mantener el mismo número de nodos negros en los caminos desde la raíz hasta las hojas nulas.

4.2.1. Estado inicial del árbol

Antes de comenzar las eliminaciones se toma como referencia el árbol rojinegro obtenido al finalizar las diez inserciones.

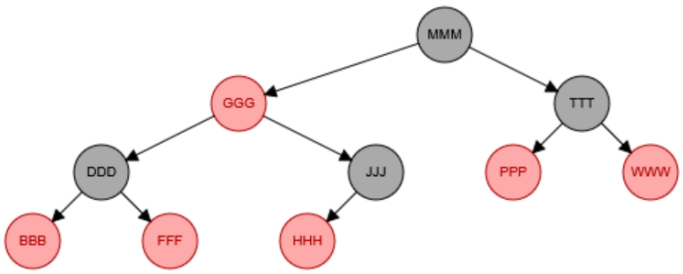


Figura 52 Árbol rojinegro inicial para el proceso de eliminación.

4.2.2. Eliminación de BBB060

La primera clave eliminada es BBB060. Este nodo corresponde a una hoja roja.

Al retirar un nodo rojo que no tiene descendientes, la cantidad de nodos negros de los caminos no cambia, por lo que no es necesario realizar ningún ajuste adicional.

4.2.3. Eliminación de DDD125

La siguiente clave es DDD125. En este estado del árbol, DDD125 es un nodo negro que tiene como único hijo a FFF190, que es rojo.

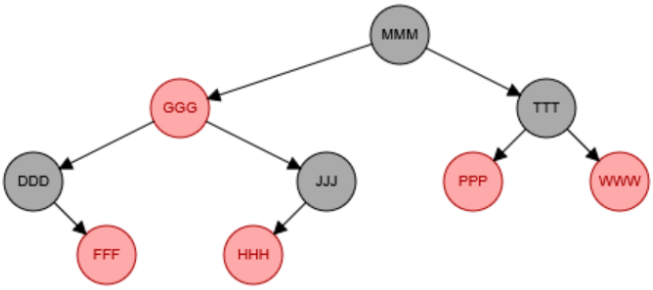


Figura 53 Árbol rojinegro después de eliminar `BBB060`.

El hijo ocupa entonces la posición del nodo eliminado y cambia su color a negro. De esta manera se conserva la cantidad de nodos negros presente en los caminos de ese subárbol.

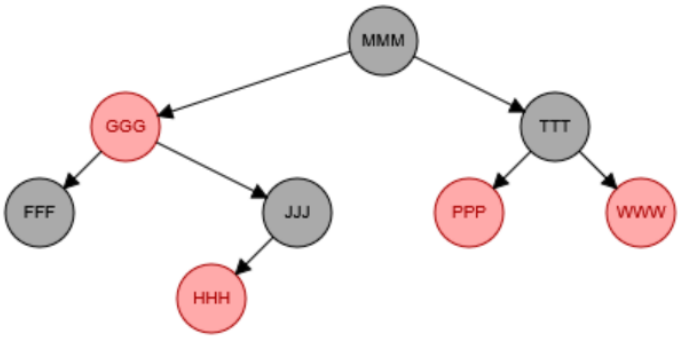


Figura 54 Árbol rojinegro después de eliminar `DDD125`.

4.2.4. Eliminación de `GGG250`

La clave `GGG250` tiene dos hijos. Para realizar su eliminación se utiliza el predecesor `FFF190`.

El nodo `FFF190` pasa a ocupar la posición que tenía `GGG250` y adopta el color correspondiente a esa posición.

Después del reemplazo aparece un desbalance en la cantidad de nodos negros de los caminos del subárbol. Para corregir esta situación se realiza una rotación doble derecha–izquierda acompañada de los recoloreos necesarios.

4.2.5. Primera rotación

El primer paso consiste en realizar una rotación a la derecha sobre `JJJ375`.

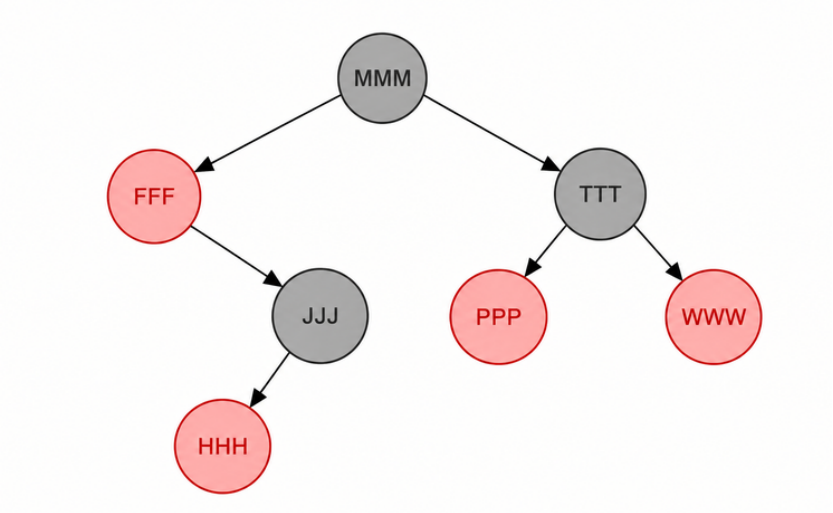


Figura 55 Árbol rojinegro después de reemplazar `GGG250` por su predecesor `FFF190`.

Con esta operación, `HHH310` asciende y `JJJ375` pasa a ocupar la posición de hijo derecho dentro del subárbol.

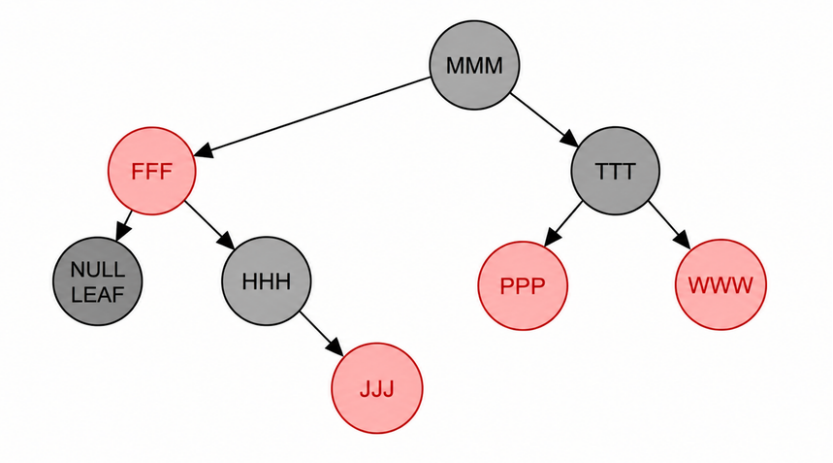


Figura 56 Árbol rojinegro después de la rotación a la derecha sobre `JJJ375`.

Durante este ajuste se mantiene visible la hoja nula que presenta el déficit de altura negra, ya que todavía falta completar la corrección.

4.2.6. Segunda rotación y recoloreo

A continuación se realiza una rotación a la izquierda sobre `FFF190`.

Después de este movimiento, `HHH310` queda como raíz del subárbol, mientras que `FFF190` y `JJJ375` quedan como hijos izquierdo y derecho respectivamente.

También se ajustan los colores de los nodos para recuperar las propiedades del árbol rojinegro.

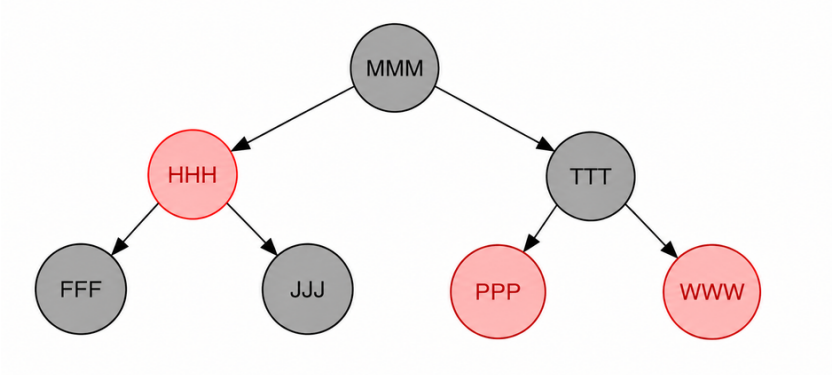


Figura 57 Árbol rojinegro después de la rotación a la izquierda sobre `FFF190` y el recoloreo.

Con esta segunda rotación se completa la corrección originada por la eliminación de `GGG250`.

4.2.7. Eliminación de `TTT750`

La siguiente clave eliminada es `TTT750`. Este nodo tiene dos hijos, por lo que se reemplaza por su predecesor `PPP625`.

El nodo que ocupa la nueva posición adopta el color negro y `WWW875` permanece como hijo derecho.

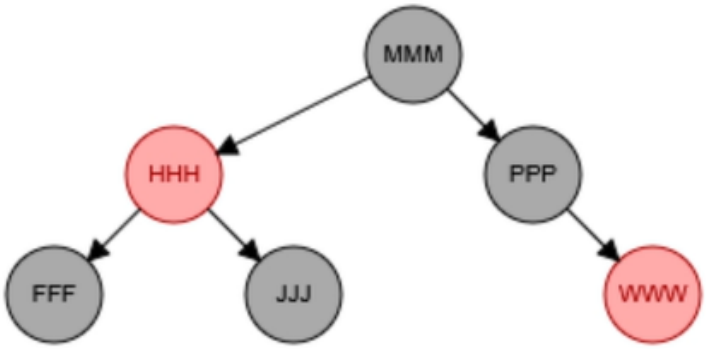


Figura 58 Árbol rojinegro después de eliminar `TTT750`.

Después de este reemplazo las propiedades del árbol se conservan sin necesidad de nuevas rotaciones.

4.2.8. Eliminación de `MMM500`

La última clave eliminada es `MMM500`, que corresponde a la raíz.

Como tiene dos hijos, se utiliza su predecesor `JJJ375`, que pasa a ocupar la posición de la raíz.

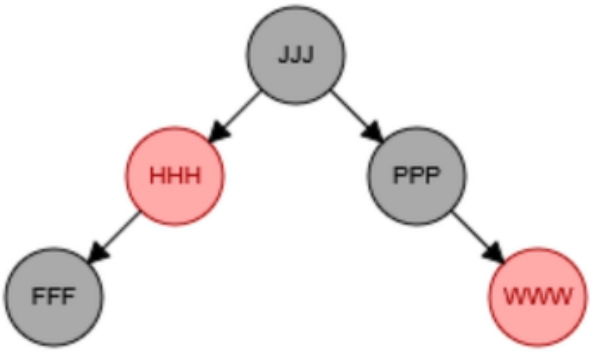


Figura 59 Árbol rojinegro después de reemplazar `MMM500` por su predecesor `JJJ375`.

Después del reemplazo, los caminos desde la raíz hasta las hojas nulas no conservan inicialmente la misma cantidad de nodos negros.

Para recuperar esta propiedad se realizan recoloreos sobre los nodos afectados.

El árbol final contiene las siguientes claves:

```
FFF190, HHH310, JJJ375, PPP625, WWW875
```

Durante las eliminaciones fue necesario realizar una rotación doble derecha–izquierda después de eliminar `GGG250`, además de varios recoloreos para conservar las propiedades del árbol rojinegro.

El recorrido inorden de la estructura final produce la misma secuencia obtenida en el AVL, lo que permite comprobar nuevamente que ambas estructuras conservan los mismos elementos.

4.3. Eliminaciones en el montículo máximo

Para el montículo se parte de la estructura obtenida después de las diez inserciones del Punto 7. En este caso, las eliminaciones no siguen las mismas reglas de los árboles binarios de búsqueda, ya que el montículo debe conservar la propiedad de prioridad entre padres e hijos y, además, mantener la forma de árbol completo.

Las cinco claves se eliminan en el mismo orden utilizado para las otras estructuras:

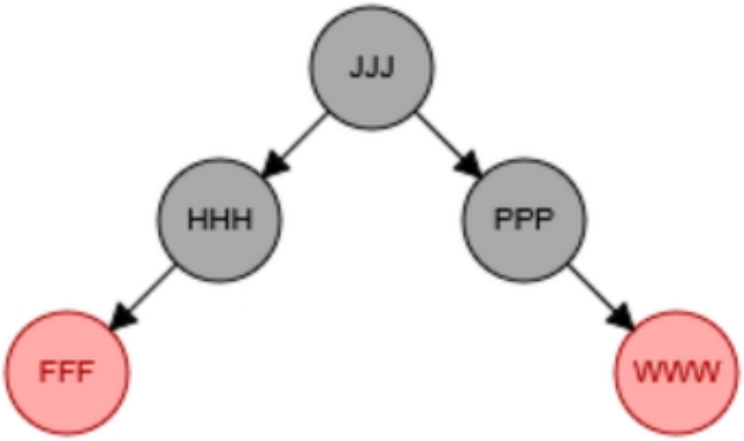


Figura 60 Árbol rojinegro final después de las cinco eliminaciones.

`BBB060, DDD125, GGG250, TTT750, MMM500`

4.3.1. Eliminación de `BBB060`

La clave `BBB060` se encuentra en una hoja del montículo. Al no tener descendientes, puede eliminarse sin producir cambios adicionales en la estructura.

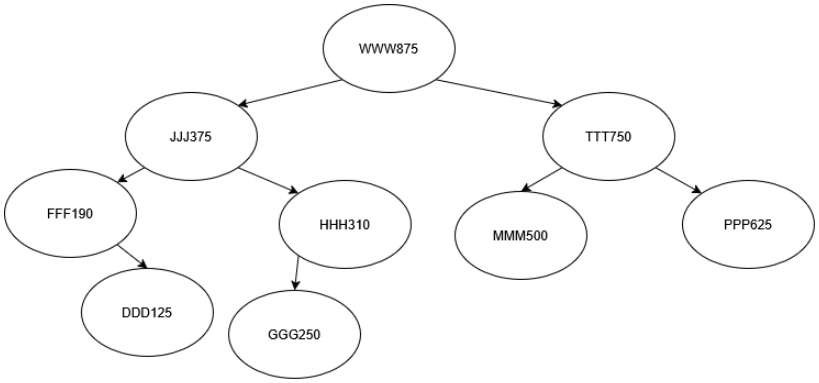


Figura 61 Montículo después de eliminar `BBB060`.

Después de esta eliminación la propiedad del montículo máximo continúa cumpliéndose.

4.3.2. Eliminación de `DDD125`

La siguiente clave eliminada es `DDD125`. Este elemento también se encuentra en una hoja del montículo.

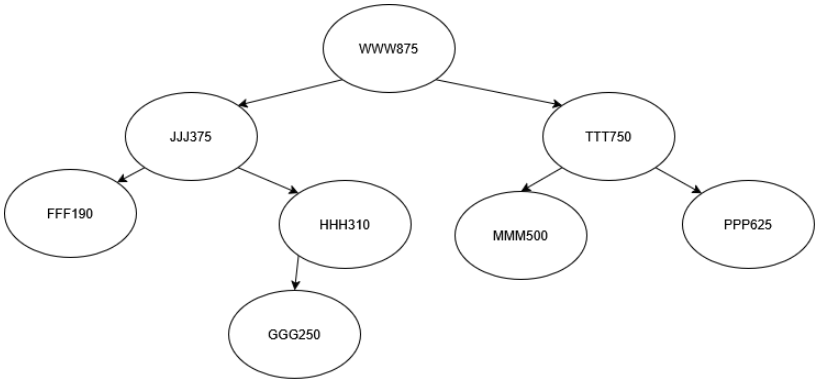


Figura 62 Montículo después de eliminar `DDD125`.

Como el nodo eliminado no tiene hijos, la relación de prioridad entre los elementos restantes no se modifica.

4.3.3. Eliminación de `GGG250`

La tercera clave corresponde a `GGG250`. En el estado actual del montículo este elemento también aparece como una hoja.

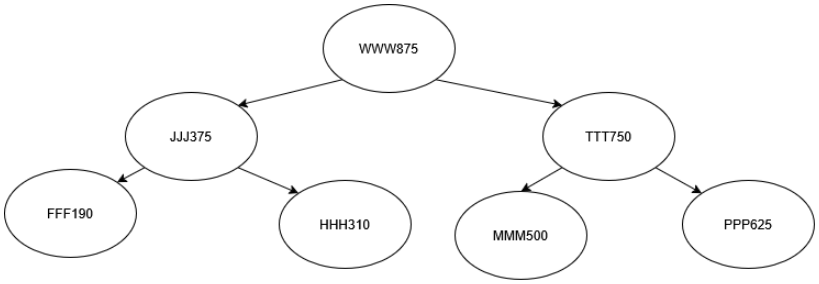


Figura 63 Montículo después de eliminar `GGG250`.

Por esta razón, las primeras tres eliminaciones pueden realizarse sin ajustes adicionales en la propiedad del montículo.

4.3.4. Eliminación de `TTT750`

La situación cambia con `TTT750`, ya que se trata de un nodo interno del montículo. En el desarrollo gráfico, su posición es ocupada inicialmente por `MMM500`.

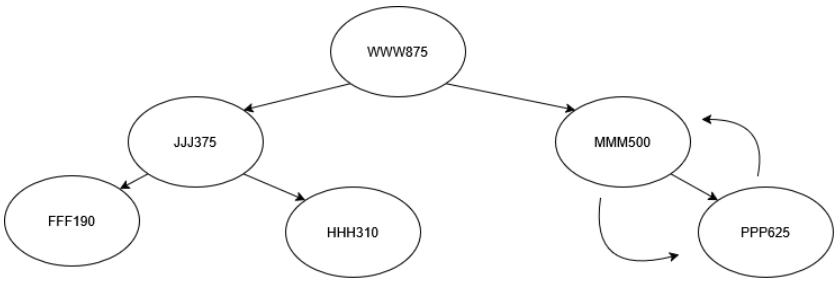


Figura 64 Montículo después de eliminar `TTT750`.

Después de este reemplazo aparece una violación de la propiedad del montículo: `MMM500` queda como padre de `PPP625`, aunque `PPP625` es mayor.

Para recuperar la propiedad del montículo máximo se intercambian estos dos elementos.

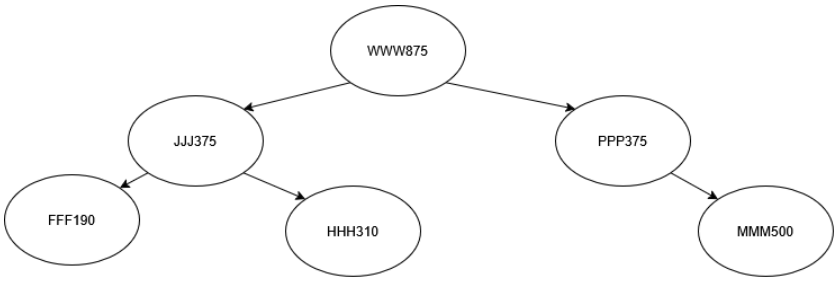


Figura 65 Montículo después del intercambio entre `MMM500` y `PPP625`.

Después del intercambio, `PPP625` ocupa la posición superior y `MMM500` desciende, recuperándose la relación de prioridad.

4.3.5. Eliminación de `MMM500`

La última clave de la secuencia es `MMM500`. Después del ajuste anterior este elemento se encuentra en una hoja, por lo que puede retirarse sin producir nuevas reorganizaciones.

Después de completar las cinco eliminaciones permanecen los siguientes elementos:

```
FFF190, HHH310, JJJ375, PPP625, WWW875
```

Durante este proceso, las primeras tres claves pudieron retirarse sin realizar ajustes adicionales. Al eliminar `TTT750` fue necesario realizar un intercambio entre `MMM500` y `PPP625` para recuperar la propiedad del montículo.

Al igual que en el Punto 7, la organización interna del montículo no corresponde a una secuencia ascendente. Para poder comparar el resultado con los árboles AVL y rojinegro se utiliza nuevamente la salida ordenada.

La secuencia ascendente obtenida es:

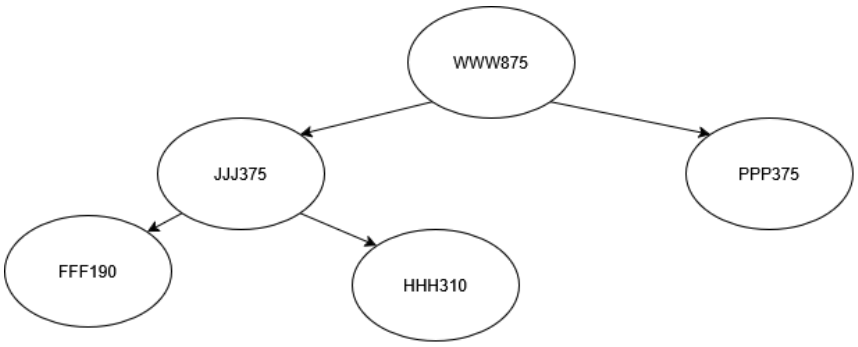


Figura 66 Montículo máximo final después de las cinco eliminaciones.

```
FFF190, HHH310, JJJ375, PPP625, WWW875
```

Esta secuencia coincide con los recorridos inorden obtenidos en los otros dos árboles, confirmando que después de las cinco eliminaciones las tres estructuras conservan exactamente el mismo conjunto de claves.

4.4. Resumen de los ajustes estructurales

Los desarrollos gráficos permiten observar que las tres estructuras procesan las mismas claves, pero requieren ajustes diferentes para conservar sus propiedades internas.

Cuadro 1 Resumen de los ajustes realizados durante los Puntos 7 y 8.

Operación	AVL	Rojinegro	Montículo
10 inserciones	Sin rotaciones	2 recoloreos	7 intercambios
5 eliminaciones	1 rotación doble	1 rotación doble y recoloreos	1 intercambio

A pesar de estas diferencias estructurales, después de las diez inserciones y también después de las cinco eliminaciones las tres estructuras producen el mismo conjunto ordenado de elementos. Esto coincide con la comprobación realizada posteriormente por el programa.

5. Medición de tiempos

Después de verificar que las tres estructuras producían los mismos resultados, se realizaron pruebas de tiempo para comparar su comportamiento durante las operaciones de inserción y eliminación.

Para cada archivo evaluado se realizaron cinco ejecuciones independientes, utilizando la misma configuración y comenzando cada prueba con las estructuras vacías. En cada ejecución se registraron los tiempos acumulados de las operaciones `add` y `del`.

Los resultados individuales se conservaron como evidencia y posteriormente se calculó el promedio de las cinco ejecuciones para cada estructura.

5.1. Mediciones individuales de inserción

La Tabla 2 presenta los tiempos acumulados de `add` obtenidos en cada una de las cinco ejecuciones realizadas para los archivos 01 a 04. Todas las mediciones se expresan en milisegundos.

Cuadro 2 Tiempos de las cinco ejecuciones para las pruebas de inserción.

Archivo	Ejecución	AVL (ms)	Rojinegro (ms)	Montículo (ms)
01	1	0.027	0.009	0.005
01	2	0.014	0.009	0.005
01	3	0.014	0.009	0.005
01	4	0.014	0.009	0.006
01	5	0.014	0.009	0.005
02	1	0.174	0.079	0.048
02	2	0.201	0.069	0.054
02	3	0.213	0.072	0.057
02	4	0.189	0.068	0.052
02	5	0.188	0.073	0.053
03	1	9.950	0.924	0.578
03	2	11.193	0.832	0.499
03	3	10.026	0.864	0.593
03	4	10.322	0.871	0.572
03	5	7.885	0.860	0.587
04	1	106681.73	54.519	24.355
04	2	107221.31	54.184	25.107
04	3	106642.62	53.492	24.362
04	4	107002.06	53.231	24.397
04	5	106829.81	55.084	24.558

El archivo 05, correspondiente a 1000000 de inserciones, se intentó ejecutar, pero la prueba superó una hora sin finalizar. Por esta razón no fue posible obtener cinco mediciones completas para este archivo.

5.2. Tiempos promedio de inserción

Para analizar las inserciones se utilizaron los archivos de únicamente operaciones `add` con tamaños de 10, 100, 1000 y 100000 elementos.

La Tabla 3 presenta los tiempos acumulados promedio obtenidos.

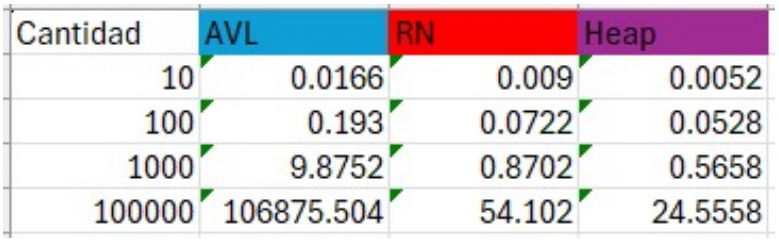
Como respaldo del cálculo realizado en Excel, también se conserva la tabla de promedios obtenida a partir de las cinco ejecuciones de cada archivo.

La Figura 68 muestra la comparación gráfica de estos resultados.

Debido a la diferencia considerable entre el tiempo del AVL para 100000 inserciones y los tiempos obtenidos por las otras dos estructuras, se utilizó una escala logarítmica

Cuadro 3 Tiempos acumulados promedio de inserción.

Inserciones	AVL (ms)	Rojinegro (ms)	Montículo (ms)
10	0.0166	0.0090	0.0052
100	0.1930	0.0722	0.0528
1000	9.8752	0.8702	0.5658
100000	106875.5042	54.1020	24.5558



Cantidad	AVL	RN	Heap
10	0.0166	0.009	0.0052
100	0.193	0.0722	0.0528
1000	9.8752	0.8702	0.5658
100000	106875.504	54.102	24.5558

Figura 67 Tabla de promedios utilizada para las pruebas de inserción.

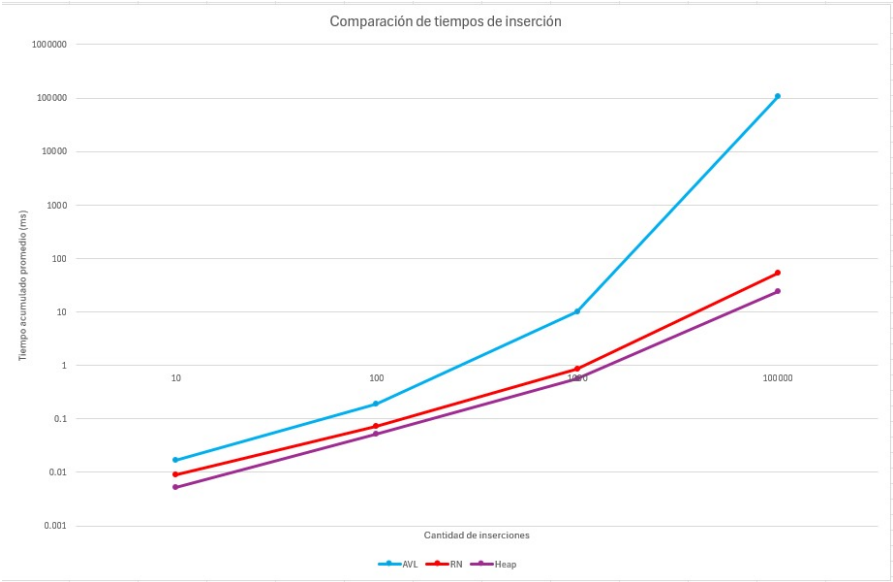


Figura 68 Comparación de los tiempos promedio de inserción.

en el eje vertical. Esto permite observar de forma más clara las tres series dentro de una misma gráfica.

5.3. Punto 9: análisis de las inserciones

En las pruebas de inserción, el montículo presentó los menores tiempos acumulados promedio para todos los tamaños evaluados. Después se ubicó el árbol rojinegro, mientras que el AVL mostró un crecimiento mucho mayor a medida que aumentó la cantidad de elementos.

5.3.1. a. Efecto del volumen de datos

Para 10 inserciones, los tiempos promedio fueron de aproximadamente `0.0166 ms` para AVL, `0.0090 ms` para rojinegro y `0.0052 ms` para montículo. En este tamaño las diferencias todavía son pequeñas.

Con 1000 inserciones, el AVL alcanzó un tiempo promedio de `9.8752 ms`, mientras que el rojinegro y el montículo se mantuvieron en `0.8702 ms` y `0.5658 ms`, respectivamente.

La diferencia se vuelve mucho más evidente con 100000 inserciones. En este caso, el AVL obtuvo un tiempo promedio de `106875.5042 ms`, frente a `54.1020 ms` del árbol rojinegro y `24.5558 ms` del montículo.

El comportamiento observado en el AVL puede estar relacionado con la forma en que está implementado el balanceo. Durante estas operaciones se calculan las alturas de los subárboles de forma recursiva. A medida que aumenta el tamaño de la estructura, este trabajo adicional puede contribuir al crecimiento del tiempo observado.

En contraste, el árbol rojinegro utilizado mediante `std::set` mantiene tiempos considerablemente menores en las pruebas realizadas. El montículo también presenta tiempos bajos durante las inserciones, siendo la estructura más rápida en todos los tamaños evaluados.

También se intentó ejecutar el archivo correspondiente a 1000000 de inserciones. Sin embargo, la ejecución superó una hora sin finalizar, por lo que se detuvo la prueba. Para el análisis se tomó entonces 100000 como el tamaño máximo ejecutado de forma razonable en el equipo utilizado.

La gráfica permite observar claramente esta tendencia. El crecimiento del AVL es mucho mayor que el presentado por las otras estructuras, mientras que el rojinegro y el montículo mantienen tiempos comparativamente bajos para los tamaños evaluados.

5.3.2. b. Cambio de ventaja entre estructuras

En los tamaños que pudieron ejecutarse completamente no se observó un cambio de ventaja entre las estructuras. El montículo presentó los menores tiempos promedio de inserción en 10, 100, 1000 y 100000 elementos, seguido por el árbol rojinegro y posteriormente por el AVL. Por lo tanto, las mediciones disponibles no muestran un intervalo en el que otra estructura pase a superar al montículo. Para el caso de 1000000 de inserciones no se puede establecer una comparación, ya que la ejecución no logró finalizar.

5.4. Mediciones individuales de eliminación

La Tabla 4 presenta los tiempos acumulados de `del` obtenidos en cada una de las cinco ejecuciones realizadas para los archivos 06 a 09. Todas las mediciones se expresan en milisegundos.

Cuadro 4 Tiempos de las cinco ejecuciones para las pruebas de eliminación.

Archivo	Ejecución	AVL (ms)	Rojinegro (ms)	Montículo (ms)
06	1	0.008	0.009	0.023
06	2	0.008	0.008	0.021
06	3	0.008	0.007	0.020
06	4	0.007	0.008	0.021
06	5	0.008	0.009	0.022
07	1	0.007	0.005	0.016
07	2	0.006	0.004	0.016
07	3	0.006	0.005	0.017
07	4	0.005	0.004	0.015
07	5	0.006	0.005	0.015
08	1	0.094	0.066	0.744
08	2	0.139	0.095	1.146
08	3	0.061	0.044	0.461
08	4	0.064	0.042	0.466
08	5	0.103	0.072	0.812
09	1	4.601	0.939	43.168
09	2	1.755	0.341	36.775
09	3	2.335	0.373	37.732
09	4	1.992	0.359	37.547
09	5	1.761	0.354	36.932

El archivo 10, correspondiente a un millón de inserciones y un millón de eliminaciones, se intentó ejecutar, pero su tiempo de ejecución excedió un tiempo razonable para las pruebas en el equipo utilizado. Por esta razón no fue posible obtener cinco mediciones completas para este archivo.

5.5. Tiempos promedio de eliminación

Para evaluar las eliminaciones se utilizaron los archivos que combinan inserciones y eliminaciones. Se registró únicamente el tiempo acumulado correspondiente a las operaciones `del`.

Se evaluaron los archivos 06, 07, 08 y 09, correspondientes a casos con 10, 5, 100 y 1000 eliminaciones, respectivamente. El archivo 10 no se incluyó en las mediciones finales debido al tiempo de ejecución requerido para procesar un millón de inserciones y un millón de eliminaciones en el equipo utilizado.

La Figura 69 presenta la comparación gráfica de los tiempos promedio obtenidos para los cuatro archivos evaluados.

Debido al crecimiento observado en el archivo 09, especialmente para el montículo, los valores de los casos pequeños aparecen visualmente cercanos al eje horizontal. La

Cuadro 5 Tiempos acumulados promedio de eliminación.

Caso	AVL (ms)	Rojinegro (ms)	Montículo (ms)
06 (10 del)	0.0078	0.0082	0.0214
07 (5 del)	0.0060	0.0046	0.0158
08 (100 del)	0.0922	0.0638	0.7258
09 (1000 del)	2.4888	0.4732	38.4308

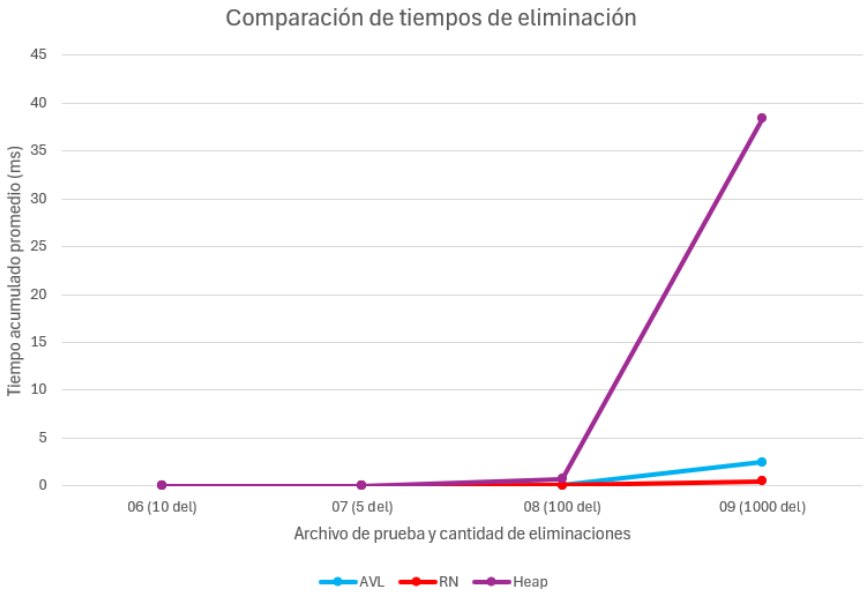


Figura 69 Comparación de los tiempos promedio de eliminación para los archivos 06 a 09.

gráfica se mantiene en escala lineal para conservar una comparación directa de los tiempos medidos.

5.6. Punto 10: análisis de las eliminaciones

En las pruebas de eliminación, el árbol rojinegro presentó los menores tiempos promedio en la mayoría de los casos evaluados. El AVL mantuvo tiempos bajos para los archivos pequeños, mientras que el montículo presentó los mayores tiempos, especialmente al aumentar la cantidad de operaciones.

5.6.1. c. Efecto del tamaño y de la secuencia de operaciones

En el archivo 06, con diez eliminaciones, los tiempos promedio fueron de `0.0078 ms` para AVL, `0.0082 ms` para rojinegro y `0.0214 ms` para montículo. En este caso, AVL fue ligeramente más rápido que el árbol rojinegro.

En el archivo 07, con cinco eliminaciones, los tiempos fueron de `0.0060 ms` para AVL, `0.0046 ms` para rojinegro y `0.0158 ms` para montículo. En este caso, el árbol rojinegro presentó el menor tiempo.

Para el archivo 08, con cien eliminaciones, los tiempos aumentaron a `0.0922 ms` para AVL, `0.0638 ms` para rojinegro y `0.7258 ms` para montículo. El árbol rojinegro continuó presentando el menor tiempo, mientras que el montículo mostró un crecimiento mayor.

En el archivo 09, con mil eliminaciones, los tiempos promedio fueron de `2.4888 ms` para AVL, `0.4732 ms` para rojinegro y `38.4308 ms` para montículo. En este caso se observa con mayor claridad la diferencia de desempeño entre las tres estructuras.

Además del número de eliminaciones, la secuencia de operaciones también cambia entre los archivos evaluados. En los archivos 06 y 07 se realizan primero las diez inserciones y después las eliminaciones. En cambio, en los archivos 08 y 09 las operaciones de inserción y eliminación aparecen intercaladas. Esto hace que las eliminaciones se realicen mientras la estructura continúa cambiando y con diferentes cantidades de elementos almacenados. En el archivo 08 llegaron a mantenerse hasta 36 elementos simultáneamente y en el archivo 09 hasta 367.

Por esta razón, el aumento de los tiempos observado en los archivos 08 y 09 no debe atribuirse únicamente al número de eliminaciones. También cambian la cantidad de elementos presentes durante las operaciones y la forma en que se intercalan las inserciones y eliminaciones. Con estas pruebas no es posible separar completamente el efecto de cada uno de estos factores, pero sí se observa que los casos de mayor tamaño presentan diferencias más marcadas entre las estructuras.

5.6.2. d. Cambio de ventaja entre estructuras

El cambio de ventaja entre AVL y rojinegro no fue completamente consistente en el archivo 06. En las cinco ejecuciones hubo casos en los que AVL fue más rápido, uno en el que ambos obtuvieron el mismo tiempo registrado y otro en el que el rojinegro fue más rápido. En el archivo 07, en cambio, el rojinegro presentó un menor tiempo de eliminación en las cinco ejecuciones. Este mismo comportamiento se mantuvo en los archivos 08 y 09, donde el rojinegro fue más rápido que el AVL en todas las mediciones realizadas. Por lo tanto, a partir de los casos de mayor tamaño evaluados, la ventaja del árbol rojinegro sí aparece de forma consistente.

El comportamiento del montículo puede estar relacionado con la estrategia utilizada para eliminar elementos. En la implementación realizada, primero se localiza el elemento dentro de la estructura, luego se elimina y finalmente se vuelve a organizar el montículo utilizando `make_heap`. A medida que aumenta la cantidad de datos y eliminaciones, este procedimiento requiere un trabajo adicional que se refleja en los tiempos medidos.

También se observa una diferencia importante respecto a las pruebas de inserción. Aunque el montículo presentó tiempos bajos al insertar elementos, en las eliminaciones fue la estructura con los mayores tiempos acumulados promedio.

El archivo 10 no se incluyó en las mediciones finales debido al tiempo de ejecución requerido para procesar un millón de inserciones y un millón de eliminaciones en el equipo utilizado.

Al finalizar las operaciones de los archivos evaluados, las tres estructuras conservaron resultados equivalentes en cuanto a los elementos procesados, por lo que las diferencias observadas corresponden al desempeño de las implementaciones y no a conjuntos de datos distintos.

6. Conclusiones

El desarrollo del taller permitió comparar tres estructuras que pueden almacenar el mismo conjunto de elementos, pero que utilizan mecanismos diferentes para mantener su organización interna. El árbol AVL, el árbol rojinegro y el montículo procesaron las mismas operaciones de inserción y eliminación y, al finalizar las pruebas, produjeron conjuntos de datos equivalentes.

El desarrollo gráfico permitió observar con mayor claridad estas diferencias. En el AVL el equilibrio se mantiene mediante rotaciones cuando la diferencia de alturas de los subárboles supera el rango permitido. En el árbol rojinegro se utilizan principalmente cambios de color y, cuando es necesario, rotaciones. En el montículo los ajustes se realizan mediante intercambios para conservar la relación de prioridad entre padres e hijos.

Para la secuencia de diez inserciones utilizada en el taller, el AVL no necesitó realizar rotaciones, mientras que el árbol rojinegro requirió dos recoloreos y el montículo realizó varios intercambios. Durante las cinco eliminaciones, tanto el AVL como el árbol rojinegro necesitaron una rotación doble derecha–izquierda, mientras que el montículo requirió un intercambio para recuperar su propiedad.

Las pruebas funcionales también permitieron comprobar que la organización interna de las estructuras puede ser diferente sin que esto implique almacenar datos distintos. Después de ordenar las salidas, las tres estructuras presentaron las mismas claves tanto después de las inserciones como después de las eliminaciones.

En las pruebas de rendimiento, el montículo presentó los menores tiempos de inserción para todos los tamaños evaluados, seguido por el árbol rojinegro. El AVL mostró un aumento considerable del tiempo al crecer la cantidad de elementos, especialmente en la prueba de 100000 inserciones. Este resultado puede estar relacionado con las decisiones tomadas en la implementación propia del AVL, particularmente con los cálculos recursivos de altura utilizados durante el proceso de balanceo.

En las eliminaciones se observó un comportamiento diferente. En los casos pequeños, AVL y árbol rojinegro presentaron tiempos relativamente cercanos, pero al aumentar la cantidad de operaciones el árbol rojinegro mostró menores tiempos que el AVL. El montículo obtuvo los mayores tiempos en todos los casos evaluados, con una diferencia especialmente marcada en el archivo 09. Esto muestra que una estructura que presenta un buen comportamiento para una operación no necesariamente presenta el mismo resultado para otra.

7. Uso de herramientas de inteligencia artificial

Durante el desarrollo del taller se utilizó ChatGPT como herramienta de apoyo. Su uso estuvo orientado principalmente a revisar la organización del informe, facilitar el uso de la plantilla, mejorar la redacción de explicaciones realizadas por el grupo,

revisar la coherencia entre los resultados obtenidos y las gráficas, y ayudar a organizar las evidencias generadas durante las pruebas.

También se utilizó como apoyo para comprender parte de la implementación de las rotaciones del árbol AVL, debido a que algunos conceptos necesarios para realizar esta parte no se encontraban completamente claros.

Algunos ejemplos de instrucciones utilizadas durante este proceso fueron:

> "Ayúdame a organizar esta explicación con un lenguaje natural de estudiante, sin cambiar los resultados obtenidos en las pruebas."

> "Ayúdame a estructurar en el informe el desarrollo paso a paso de las inserciones y eliminaciones utilizando las evidencias y diagramas que ya realizamos."

El contenido generado con apoyo de esta herramienta fue revisado y adaptado al desarrollo realizado por el grupo y a los resultados obtenidos durante la ejecución del programa.

Referencias

[1] Pontificia Universidad Javeriana: Taller 3: Árboles de búsqueda. Departamento de Ingeniería de Sistemas, Estructuras de Datos (2026)

[2] Castro Martínez, A.: Árboles Binarios. Material de clase, Estructuras de Datos, Pontificia Universidad Javeriana (2026)

[3] Castro Martínez, A.: Árboles AVL. Material de clase, Estructuras de Datos, Pontificia Universidad Javeriana (2026)

[4] Castro Martínez, A.: Árboles RN (Rojo-Negro). Material de clase, Estructuras de Datos, Pontificia Universidad Javeriana (2026)

[5] Castro Martínez, A.: Montículos. Material de clase, Estructuras de Datos, Pontificia Universidad Javeriana (2026)

[6] Galles, D.: Red-Black Tree Visualization. https://www.cs.usfca.edu/~galles/visualization/RedBlack.html. University of San Francisco, visualizador interactivo de árboles rojinegros

[7] cplusplus.com: std::set. https://cplusplus.com/reference/set/set/. Referencia de la biblioteca estándar de C++

[8] cppreference.com: std::make_heap. https://en.cppreference.com/w/cpp/algorithm/make_heap. Referencia de algoritmos de montículos de la biblioteca estándar de C++

[9] cppreference.com: std::push_heap. https://en.cppreference.com/w/cpp/algorithm/push_heap. Referencia de algoritmos de montículos de la biblioteca estándar de C++